

Göteborg Tech Week x Chalmers Hackathon x SKF

Analyze and compare learning methods suitable for industrial
humanoids

**Lahiru Ramesh
Sophearith Prum
Muhammad Ahmad Amjad**

Contents

1 Introduction	3
1.1 Challenge Background.....	3
1.2 Objective of the Project.....	3
1.3 Scope of the Work	4
1.4 Selected Task: Walking Up the Stairs	4
2 Comparison and Industrial Evaluation of Learning Methods	5
3 Generalized Humanoid Training Pipeline.....	7
4 Locomotion: Walking Up the Stairs.....	8
5 Manipulation: Pick-and-Place	10
6 Hybrid Learning Approach	12
6.1 Why a Hybrid Approach?	12
7 Implementation of Walking Up the Stairs.....	14
7.1 Simulation Environment.....	14
7.2 Robot and Inputs.....	15
7.3 PPO Training.....	16
7.4 Observation and Action Space.....	16
7.5 Reward Function	17
7.6 Curriculum Learning.....	17
7.7 Domain Randomisation.....	17
7.8 Training Configuration.....	18
7.9 Results	19
7.10 Working and Limitations.....	24
8 Implementation of Pick-and-Place: Bearing Ring	25
8.1 Task Definition.....	25
8.2 Robot Arm Stand-in.....	25
8.3 Unitree G1 with Dexterous Hands	26
8.4 Findings and Next Steps.....	27
9 SKF Skill Studio: Dashboard, Skills and Skill Composition	28
9.1 Dashboard and Source Code	28
9.2 Architecture	28
9.3 What the Dashboard Can Do.....	29
9.4 How to Define a Skill.....	37
9.5 How to Combine Skills	41
10 Challenges, Techniques and Lessons Learned	43
10.1 Challenges and How We Overcame Them.....	44
10.2 Key Techniques.....	44
10.3 What We Learned	45
11 SKF Use Case and Business Value	46
11.1 SKF Use Case: Pick-and-Place	46
11.2 Locomotion vs. Manipulation and Business KPIs	48
11.3 Business Value for SKF.....	48
11.4 Measured Results Relevant to SKF	49
12 Future Work	50
12.1 Next Technical Steps	50
12.2 Agents to Review the Process	50
12.3 Path to Real-World Deployment	51
References	52

1. Introduction

Humanoid robots have the potential to perform a wide range of tasks in industrial environments, but teaching them complex movements and behaviours remains a challenge. This hackathon task investigates different learning methods for training humanoid robots and compares their suitability for industrial applications, with a focus on locomotion and manipulation. Walking up stairs is selected as the main practical task, and a simulation-based learning pipeline is proposed and evaluated. An important part of the hackathon is also to explore the potential business value and future applications for SKF, particularly considering the large number of moving components in humanoid robots, the potential role of bearings in their design, and SKF's increasing involvement in robotics, including its recent investments related to robotics in China.

1.1 Challenge Background

The challenge focuses on how humanoid robots can be trained to perform useful tasks in industrial environments. Unlike traditional industrial robots, humanoids need to learn complex movements while maintaining balance and adapting to different situations. The hackathon therefore asks teams to investigate and compare different learning methods, including manual programming, learning from demonstration, teleoperation, imitation learning, reinforcement learning, simulation-to-real, human feedback and vision-language-action approaches. The goal is to understand the strengths and limitations of these methods and identify suitable approaches for future industrial humanoid applications.

1.2 Objective of the Project

The objective of this hackathon is to compare different learning methods, analyze locomotion and manipulation, and develop a suitable training approach for industrial humanoid robots. Based on this comparison, the team focuses on developing and evaluating a learning pipeline for humanoid locomotion.

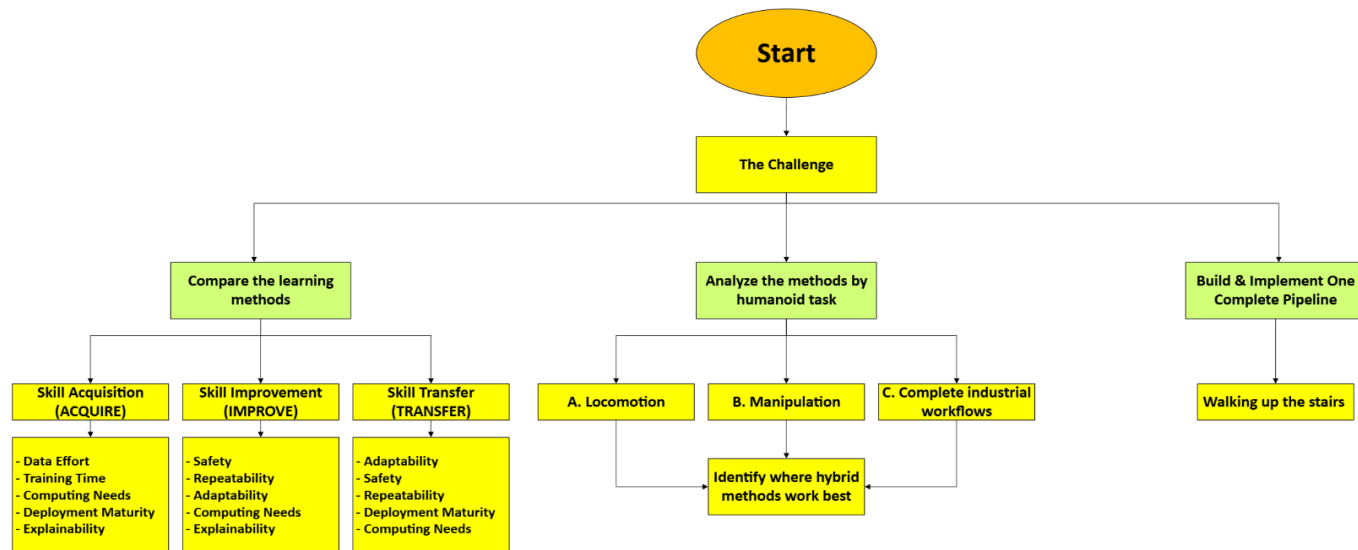


Image : Flowchart for understanding the challenge.

1.3 Scope of the Work

The work covers learning methods, industrial evaluation, locomotion and manipulation. The practical implementation focuses on walking up stairs using simulation-based reinforcement learning, while manipulation is explored through a pick-and-place use case. The project also considers safety, sim-to-real transfer and potential business value for SKF. In addition to the stair-climbing policy, a bearing-ring pick-and-drop skill was implemented in simulation, and a web platform (SKF Skill Studio) was built so that a team can define, train, evaluate and release humanoid skills.

1.4 Selected Task: Walking Up the Stairs

Walking up stairs is selected as the main practical locomotion task because it is more challenging than flat-ground walking. The humanoid must maintain balance, place its feet correctly, and adapt to the change in height between steps. This makes stair climbing a useful task for evaluating learning-based locomotion in simulation.

2. Comparison and Industrial Evaluation of Learning Methods

Different learning methods can be used to teach humanoid robots new skills. Each method has different requirements and advantages depending on the task and the industrial application. The methods are first described briefly and then evaluated using criteria such as data effort, training time, safety, adaptability and deployment maturity.

Table 1: Learning Methods

Learning Methods	Description
Manual Programming	Human explicitly programs the robot's actions and task sequence.
Learning from Demonstration (LfD)	Robot learns a skill from examples demonstrated by a human.
Teleoperation	Human directly controls the robot, often remotely, to perform a task.
Imitation Learning	A model learns a policy by imitating demonstrated behavior.
Reinforcement Learning (RL)	Robot learns through trial and error using rewards or penalties.
Simulation-to-Real	Robot is trained in simulation and the learned behavior is transferred to the real robot.
Human Feedback	Human evaluations, corrections, or preferences guide the robot's learning or improvement.
Vision-Language-Action (VLA)	Model connects visual and language information to generate robot actions.

While the learning methods differ in how the robot learns, their suitability for industrial use also depends on practical factors. Therefore, the methods are evaluated using the following industrial criteria.

Table 2: Industrial Requirements

Industrial Requirement	Description
Data effort	How much data, demonstrations, or training does the method require?
Training time	How much time and effort is needed to train or prepare the robot?
Safety	What level of risk is involved during training, testing, and deployment?
Repeatability	How consistently can the robot reproduce the desired behavior?
Adaptability	How well can the robot handle changes in the task or environment?
Computing Needs	How much computational power is needed for training and operation?
Explainability	How easily can humans understand why the robot produces an action?
Deployment Maturity	How proven and established is the method in real-world industrial

With the learning methods and industrial requirements defined, the next step is to evaluate how each method performs against these requirements. The industrial evaluation matrix provides a direct comparison of the methods using the defined criteria and a score from 1 (low) to 5 (high). The evaluation is informed by the literature review and the findings from the research on each learning method.

Table 3: Industrial Evaluation Matrix:

Learning Method	Data effort	Training Time	Safety	Repeat.	Adapt.	Computing needs	Explain.	Deployment Maturity
Manual Programming	•	••••	••••	•••••	••	•	•••••	•••••
LFD	•••	•••	•••	••••	••••	•••	•••	•••
Teleoperation	•••	••	•••	••	•••••	•••	•••••	••••
Imitation Learning	•••	•••	•••	••••	•••	••••	••	•••
RL	•••• •	•••••	•	••••	••••	•••••	••	•••
Sim-to-Real	••	••	••••	••••	••••	••••	•••	••••

Human Feedback	●●●	●●●	●●●●	●●●	●●●●	●●●●	●●●●	●●●
VLA	●●●●	●●●●	●●	●●●	●●●●●	●●●●●	●●	●●

Learning Method Abbreviations:

LFD = Learning from Demonstration · RL = Reinforcement Learning · VLA = Vision-Language-Action
 · Sim-to-Real = Simulation-to-Real Transfer

Industrial Matrix Abbreviations:

· Repeat. = Repeatability · Adapt. = Adaptability · Explain. = Explainability

Scoring:

● = 1 (Low) · ●● = 2 · ●●● = 3 (Medium) · ●●●● = 4 · ●●●●● = 5 (High)

3. Generalized Humanoid Training Pipeline

A humanoid robot can require different learning methods at different stages of training, depending on the task and the required level of performance. The generalized training pipeline combines these methods into a sequence that starts with defining the task and collecting data, followed by training, testing, validation, and deployment. The pipeline provides a general framework that can be adapted to different humanoid tasks, including locomotion and manipulation.

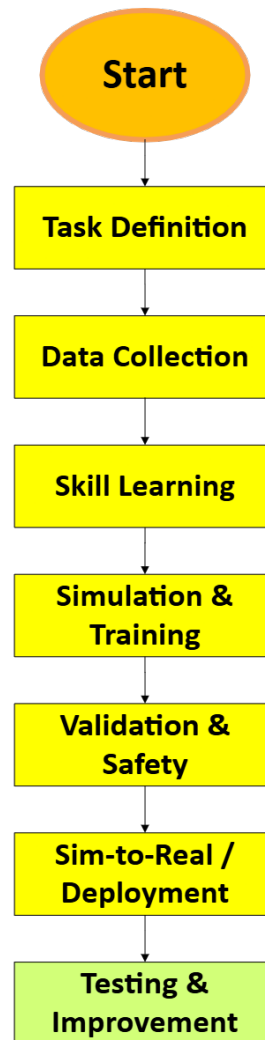


Image : Generalized Humanoid Training Pipeline

4. Locomotion: Walking Up the Stairs

Locomotion is an important capability for humanoid robots, as the robot must be able to move while maintaining balance and stability. Stair climbing is used as an example to show how different learning methods can be combined to train a humanoid for a more challenging movement task. The flowchart below presents the general learning process for stair climbing.

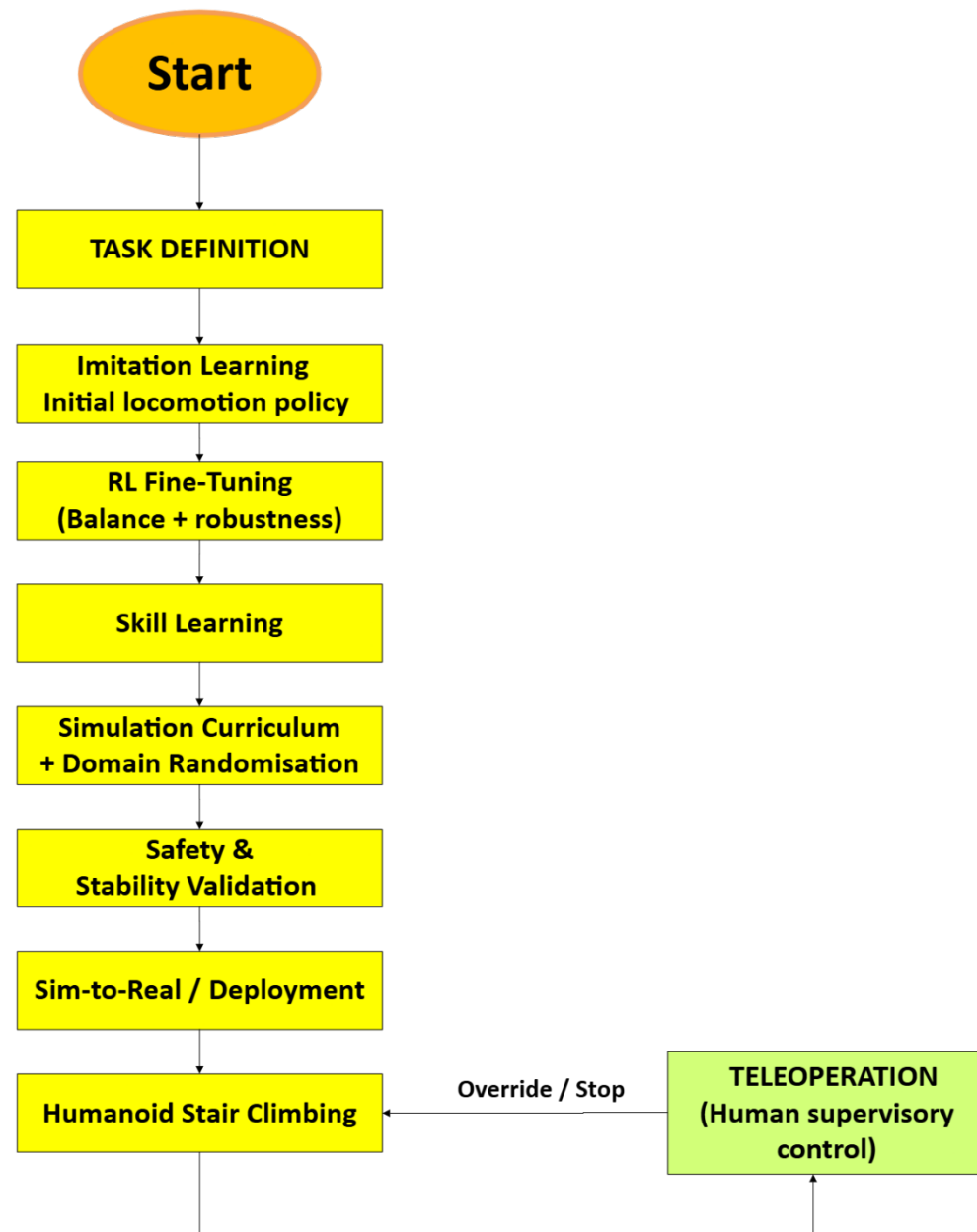


Image : Flow chart for Locomotion: Walking Up the Stairs

The locomotion training process combines several learning and validation stages. The process starts with an initial policy learned from demonstrations, followed by reinforcement learning to improve the behaviour in simulation. The trained policy is then validated and transferred to the physical humanoid, with teleoperation available as a safety intervention.

Learning Flow of the Locomotion Training Pipeline

- Imitation Learning: The robot learns an initial locomotion policy from demonstrated walking behaviour.
- RL Fine-Tuning: Reinforcement learning further optimises the locomotion policy for balance, stability, and stair-climbing performance.
- Simulation – Curriculum and Domain Randomisation: The policy is trained progressively from easier to more difficult environments while varying terrain, friction, disturbances, and other conditions.
- Safety and Stability Validation: The learned policy is evaluated against predefined stability and safety requirements before real-world deployment.
- Sim-to-Real: The validated policy is transferred from the simulation environment to the physical humanoid.
- Humanoid – Stair Climbing: The humanoid performs the staircase task under controlled real-world conditions.
- Teleoperation Feedback Loop: If unsafe or unexpected behaviour occurs, the human operator can intervene or stop the robot to prevent further execution.

5. Manipulation: Pick-and-Place

Manipulation allows a humanoid robot to interact with objects and perform useful industrial tasks. Pick-and-place is used as a simple example to show how learning methods can be applied to a manipulation task. The flowchart below presents a general training process for learning the required perception, movement, grasping, and placement actions.



Image : Flow chart for Manipulation: Pick-and-Place

Learning Flow of the Manipulation Training Pipeline

- Task Definition: Define the pick, move, and place actions, objects, constraints, and success conditions.
- Learning from Demonstration: The robot learns from examples of the manipulation task demonstrated by a human.

- Imitation Learning: The demonstrated behaviour is used to learn an initial manipulation policy.
- Initial Manipulation Policy: The robot obtains a basic policy for performing the pick-and-place task.
- Simulation & Testing: The policy is tested under different object positions, poses, and environmental conditions.
- RL Fine-Tuning: Reinforcement learning improves the policy through repeated trials and rewards.
- Safety & Stability Validation: The learned behaviour is checked for unsafe movements, collisions, and failed manipulation.
- Sim-to-Real: The validated policy is transferred from simulation to the physical humanoid.
- Physical Humanoid Testing: The robot performs the pick-and-place task under controlled real-world conditions.
- Teleoperation Override / Stop: If a problem occurs during physical testing, the operator can intervene or stop the robot.

6. Hybrid Learning Approach

6.1 Why a Hybrid Approach?

Using different learning methods together can make humanoid robot training more reliable and practical for both locomotion and manipulation tasks. Imitation Learning can provide an initial policy from human demonstrations, while Reinforcement Learning can further improve the behaviour through repeated

training and trial-and-error in simulation. Teleoperation adds human control during testing, allowing the operator to intervene or stop the robot when unexpected behaviour occurs. Combining these methods allows the strengths of different approaches to be used within the same training pipeline, while reducing the limitations of relying on a single method.

7. Implementation of Walking Up the Stairs

This chapter describes how the locomotion pipeline of Chapter 4 was implemented for the Unitree G1 humanoid. The implemented pipeline, shown below, has six stages: the skill is specified together with its success criteria, a preflight check verifies the simulator, the policy is trained, the best checkpoint is selected, the policy is qualified (certified) in a second physics engine, and only then can it be released for deployment. Every stage exists because a training run failed without it.

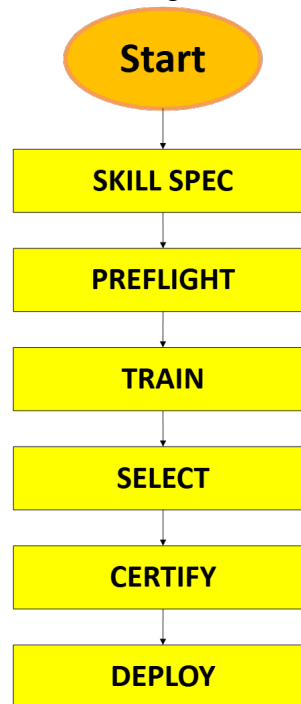


Image : Implemented stairs pipeline, from skill specification to deployment

7.1 Simulation Environment

The stair-climbing skill was developed entirely in simulation with MuJoCo Playground, an open-source robot-learning framework built on the MuJoCo physics engine. Two physics engines are used on purpose. Training runs in MuJoCo Warp/MJX on a GPU, where thousands of robots are simulated in parallel. Testing runs in the standard MuJoCo C engine on an ordinary laptop CPU. Because the two engines resolve foot contact differently, a policy that only works by exploiting one simulator is caught before it is accepted.

Training terrain: an 8×8 grid of staircases. Each row is one difficulty level, with step heights from 2 cm to 16 cm in 2 cm increments. The columns alternate between pyramids (the robot walks up and down) and pits (the robot walks down and climbs out). The tread depth is varied between 25, 29 and 33 cm so that the robot cannot memorise a single stride length.

Validation staircases: separate staircases that are never used in training: 8 step heights from 3.0 to 15.0 cm and a 27 cm tread depth that does not appear in training. All stair results in Section 7.9 come from these staircases.

Making the simulated foot contact realistic

An important finding was that the GPU engine created only one contact point between the box-shaped foot sole and the stairs, while the CPU engine created between 3 and 20. The first eight policy versions therefore learned to balance on a single point per foot. We replaced each sole with six small contact spheres, which produce the same contact in both engines. After this change the tallest step the robot could climb without a fall increased from 3 cm to 9.9 cm. A preflight check now compares the two engines on five stair poses before every training run, and the feet agree to within 0.1 cm.

7.2 Robot and Inputs

The robot is the Unitree G1 humanoid, using the open 29-joint MuJoCo model (legs, waist and arms) with the same kinematics as the hardware. Unitree's own pre-trained 12-joint walking policy is used as a baseline to show what is possible without new training.

The robot perceives the stairs through a simulated head depth camera similar to an Intel RealSense D435i, tilted 45° downwards, with a 212×120 depth image and a range of 0.3–3 m. The depth pixels are cleaned (the robot's own body is removed), projected into 3D, fused into an elevation map with 5 cm cells and sampled as 55 terrain heights around the feet, from 0.3 m behind to 1.2 m ahead of the robot. This height scan is what the policy "sees". The other inputs come from the robot's own sensors (the IMU and the joint encoders) and from the operator's walking command: speed, turning rate and step length.

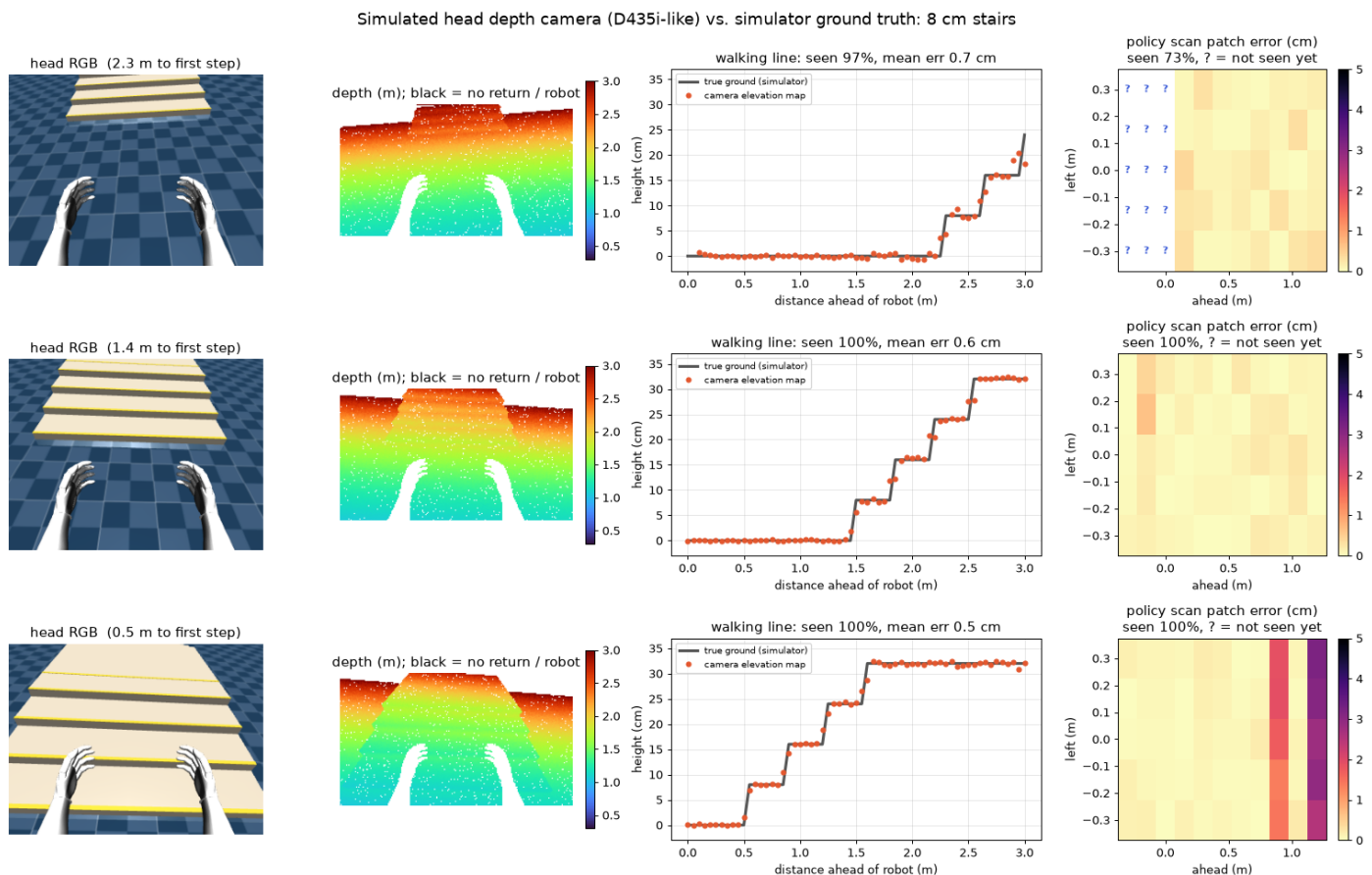


Image : Simulated head depth camera compared with the true terrain on 8 cm stairs (mean error 0.5–0.7 cm)

7.3 PPO Training

The policy is trained with Proximal Policy Optimisation (PPO), a reinforcement-learning algorithm in which the robot improves through trial and error. In each iteration, 8,192 simulated G1 robots walk in parallel for 20 control steps. The rewards they collect are used to estimate which actions were better than expected, and the neural network is updated in small, clipped steps so that learning remains stable. On one GPU this gives about 100,000 simulation steps per second, so 300 million steps, which corresponds to years of walking experience, take less than one hour.

Two techniques make training faster and more reliable:

- **Asymmetric actor–critic:** the actor, which is the part that runs on the robot, only receives realistic and noisy sensor data. The critic, which is only used during training, additionally sees privileged simulator information such as the true terrain and contact forces. This helps it judge actions more accurately without making the robot depend on information it will not have.
- **Warm start (transfer learning):** the stairs policy starts from the trained flat-ground walking policy. The new height-scan inputs start with zero weights, so at step 0 the robot walks exactly like its parent (verified: maximum action difference 0.0). Each new version then starts from the best checkpoint of the previous version, which reduced one iteration from days to about one hour.

Every training run saves 20 checkpoints. Instead of keeping the last one, each checkpoint is tested on 2,048 validation crossings on the GPU and the best one is selected, because the final checkpoint was often not the best.

7.4 Observation and Action Space

The observation is the information that the policy receives at every control step, 50 times per second. It contains 160 values, summarised in the table below.

Table 4: Observation space of the stair-climbing policy

Input group	Values	Description
Base motion	9	Base linear velocity, gyroscope and gravity direction (IMU)
Command	3	Forward speed, sideways speed and turning rate requested by the operator
Joint state	58	29 joint positions (relative to the default pose) and 29 joint velocities
Previous action	29	The last joint targets sent, which helps the policy produce smooth motion
Gait clock	4	Sine and cosine of the walking phase for each foot
Step command	2	Commanded step length and cadence (steps per second)
Height scan	55	11 × 5 terrain heights from the head camera, relative to the pelvis
Total	160	

During training, noise is added to these inputs (for example ± 1.5 rad/s on joint velocities and ± 2 cm on the height scan) so that the policy does not rely on perfect sensors. The critic receives 273 values, including the privileged information described in Section 7.3.

The action consists of 29 values, one target angle for each joint. The target equals the default standing pose plus a scaled action, and the robot's joint controllers (PD control) turn these targets into motor torques within the motor limits. The scale was widened from 0.5 rad to 1.0 rad for the hip pitch, knee and ankle pitch joints. With the original range the knee could only produce about 75 Nm when stepping up, while the motor can deliver 139 Nm, and the robot collapsed on steps higher than 8 cm.

7.5 Reward Function

The reward tells the robot what good behaviour is. We started from the tuned G1 locomotion rewards in MuJoCo Playground (following the commanded velocity, an upright posture, low torques, smooth actions, no foot slipping and sufficient foot air time) and added terms that are specific to stairs and step length:

- **Step-length reward:** at each foot touchdown the robot is rewarded for placing the foot at the commanded distance ahead of the other foot: $r = \exp(-(\Delta x - \ell)^2 / 0.05^2)$, where Δx is the achieved step and ℓ is the commanded step length. Touchdowns require at least 0.1 s of air time, so contact flicker cannot produce fake steps.
- **Foot clearance:** the swing foot is rewarded for lifting high enough to clear the next step ahead.
- **Base height:** a cost relative to the terrain keeps the pelvis at a normal walking height instead of crouching.
- **Step-edge penalty:** a cost when a foot lands on the edge of a step, applied only at touchdown. An earlier version applied it at every time step, and it dominated all other rewards.

Episodes end early when the robot falls, defined as a torso tilt above 60° or a pelvis lower than 0.45 m. Turning more than 60° away from the target direction is also counted as a fall. This closed a loophole in version 3, where the robot learned that turning away from difficult stairs gave a better score than trying to climb them.

7.6 Curriculum Learning

Instead of training on all step heights at once, a terrain curriculum is used. Each simulated robot starts on low steps. When it crosses its staircase (more than 1.9 m of progress in the commanded direction) it is promoted to the next level, and after a fall, or when it covers less than half of the commanded distance, it is demoted. This keeps every robot practising at the edge of its ability. Robots that pass the highest level are sent back to the difficult 12–16 cm rows 70 % of the time, so that training concentrates on the situations where falls happen.

The effect is large. Without a curriculum (version 1) the robot crossed only 1 of 32 validation staircases and fell in 28. With the curriculum and a warm start (version 2) falls dropped to 13 of 32, and adding hard-case replay (version 12) reduced falls on 15 cm steps from 1 in 3 to 1 in 12.

7.7 Domain Randomisation

A policy that is trained in one perfect simulation often fails when anything changes. Domain randomisation varies the physical conditions in every episode so that the policy learns to cope with a range of situations, which is essential for a later transfer to the real robot. The randomised factors are listed below.

Table 5: Domain randomisation used for the stairs policy (v14)

Factor	Range in training	Real-world reason
Floor friction	0.25 – 1.0	Dry, dusty and oily factory floors
Joint friction and armature	Varied per joint	Differences between individual motors and gearboxes
Link masses and torso payload	Varied masses, ± 1 kg on the torso	Manufacturing tolerances and carried items
Joint offsets	Small random offsets	Calibration errors
Pushes	Random shoves while walking	Contact with people or objects
Sensor noise	IMU, joints and height scan (± 2 cm)	Real sensors are never exact
Actuation delay	0 or 20 ms (one control step), in 50 % of episodes	Communication and motor latency of the real robot
Camera error model	Distance-dependent noise, 3 % dropout, blind spots	Depth-camera imperfections, simulated at GPU speed

Randomisation is a trade-off. In a controlled comparison on the step-length task, the randomised policy was about four times less precise in step length but recovered from pushes that made the non-randomised policy fall. The lesson for industry is to randomise what the real site will actually show, such as the measured floor friction, the robot's real latency and the expected pushes, rather than randomising everything "just in case".

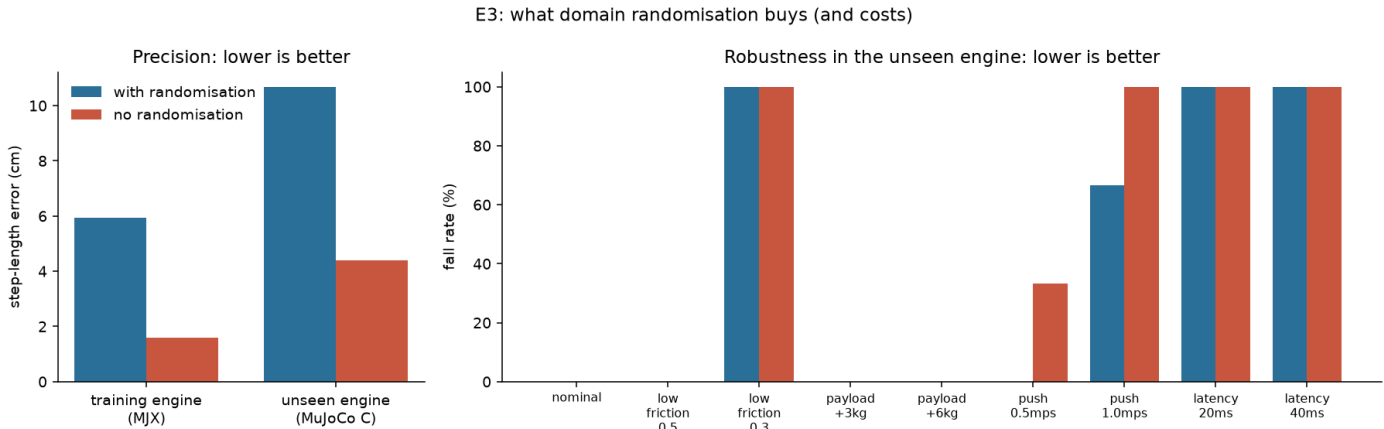


Image : Effect of domain randomisation on precision and robustness (step-length task)

7.8 Training Configuration
Functions/modalities to train the humanoid:

Table 6: Functions/modalities to train the humanoid

Component	Implementation / Function
Learning algorithm	PPO (Proximal Policy Optimization)
Policy network	MLP: 512 $\rightarrow$ 256 $\rightarrow$ 128; outputs Gaussian actions as target joint angles
Critic network	MLP: 512 $\rightarrow$ 256 $\rightarrow$ 128
Discount factor (γ)	0.97
Clip parameter (ϵ)	0.2
Entropy coefficient	0.005
Learning rate	3×10^{-4} from scratch; 1×10^{-4} when fine-tuning from a previous version
Parallel environments	8,192
Rollout / update	20 steps per environment, 32 minibatches, 4 epochs
Asymmetric actor–critic	Actor uses realistic robot sensors; critic additionally uses privileged simulation information during training
Observation normalisation	Running mean and standard deviation applied to observations
Reward shaping	Rewards velocity tracking, gait phase, foot clearance, step length and foot air time; penalises falling, slipping and low posture
Domain randomisation	Randomises friction, mass, joint offsets, pushes and sensor noise for robustness and sim-to-real transfer
Curriculum learning	Gradually increases terrain/stair difficulty based on policy performance
Transfer learning / warm start	Stair policy is initialized from a previously trained flat-walking policy
Parallel GPU simulation	MJX/Warp runs thousands of simulated robots simultaneously to accelerate training

Each training job is started with one command, or from the dashboard (Chapter 9). The job finds an available GPU machine, runs the preflight checks, trains, ranks every checkpoint and shuts the machine down when it finishes, with a hard time limit set on the machine itself. A typical iteration took about 41 minutes from launch to ranked checkpoints.

7.9 Results

The results are reported against requirements that were defined before training. All stair tests run in the second physics engine (MuJoCo C) on the validation staircases. A strict test consists of 96 runs: 8 step heights $\times$ 4 staircases $\times$ 3 starting positions.

Table 7: Requirements and results of the best stairs policy (v14)

Requirement	Target	v14 result	Status
Crossing, true terrain scan	≥ 90 % crossed, ≤ 5 falls in 96	95/96 crossed, 1 fall	Met
Qualified step height, true terrain scan	≥ 10 cm with no falls up to it	9.9 cm	Nearly met
Crossing, head camera	≥ 90 % crossed, ≤ 5 falls in 96	86/96 crossed, 8 falls	Not met
Qualified step height, head camera	≥ 10 cm	6.4 cm	Not met
Pushes, friction $\times 0.5$, 0.6 m/s, 30 cm treads	Few or no falls	0–2 falls in 32–96 runs	Mostly met
5 kg torso payload	≤ 2 % falls	4/32 fell	Not met
Extra actuation delay (40 ms in total)	≤ 2 % falls	17/32 fell	Not met
Robot safety: motor torque saturation	As low as possible	4.6 % of the time (v12: 13 %)	Improved
Release gate	All required conditions pass	Not passed	Correctly withheld

Baseline first: what is possible without training?

Unitree's pre-trained walking policy was run for 105 combinations of speed and gait timing in 15 seconds on a laptop, without a single fall. By re-timing its internal gait clock, step lengths from 7 to 40 cm were reachable at no training cost. However, speed and step length could not be controlled independently, and the blind vendor policy cannot climb steps higher than about 4 cm. This is the baseline that training has to beat.

Step-length policy on flat ground

The step-length policy was trained for 200 million steps in about 95 minutes. It walks without falling across the full grid of 20 commands in both physics engines and tracks the commanded speed within about 0.03 m/s. The step length met the 3 cm accuracy target for feasible commands at 0.7–0.9 m/s in the training engine, but not across the whole range, and the mean error grew to 10.7 cm in the second engine. The policy showed that a new controllable parameter can be added to an existing gait, and it became the starting point for the stairs policy.

Stair climbing: fourteen versions

The stairs policy was developed over 14 versions in three days. Each version responded to a measured failure of the previous one, as shown in the table below. On staircases that the robot had never trained on, falls dropped from 28 of 32 runs (v1) to 1 of 96 runs (v14).

Table 8: Development history of the stair-climbing policy

Version	What changed	Result on validation stairs	Lesson
v1	PPO from scratch, all step heights mixed	1/32 crossed, 28 fell	A curriculum is needed
v2	Terrain curriculum, warm start from flat walking	4/32 crossed, 13 fell	Walks down, stalls at the first step up
v3–v5	Heading command; turning away counts as a fall	Up to 18/32 crossed	A reward loophole was closed
v6–v7	Camera model; mixed tread depths	17/32 crossed, 14 fell	Avoid over-fitting to one tread depth
v8–v9	Wider leg action range; six contact spheres per sole	Strict: 81/96 crossed, 9 fell	Physics fidelity gave the largest gain (3 → 9.9 cm)
v10–v11	Lower learning rate; stair cadence 1.25–1.6 Hz	Strict: 88/96 crossed, 8 fell	About one step per tread
v12	Hard-case replay of 12–16 cm steps	Strict: 91/96 crossed, 5 fell	Practise where the falls are
v13	Fine-tuning on the camera error model	Camera: 83/96 crossed, 11 fell	The camera gap needs another method
v14	Actuation delay and low-friction training	Strict: 95/96, 1 fell; camera: 86/96, 8 fell	Best policy; release gate not passed

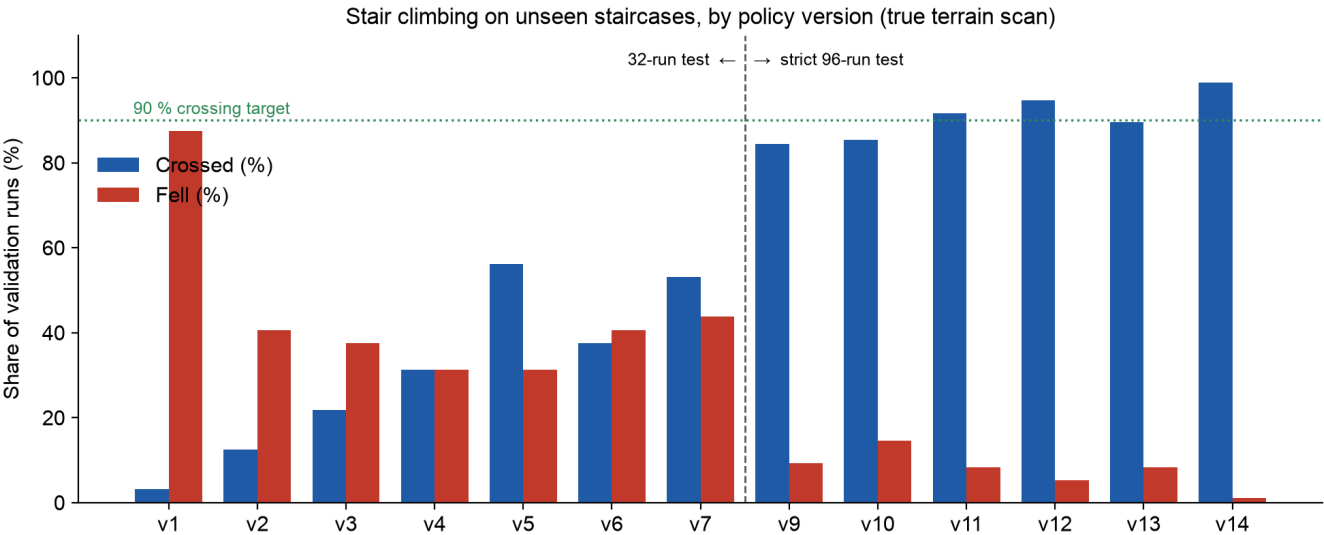


Image : Crossing and fall rates on the validation staircases for each policy version (v1–v7: 32-run test, v9–v14: strict 96-run test)

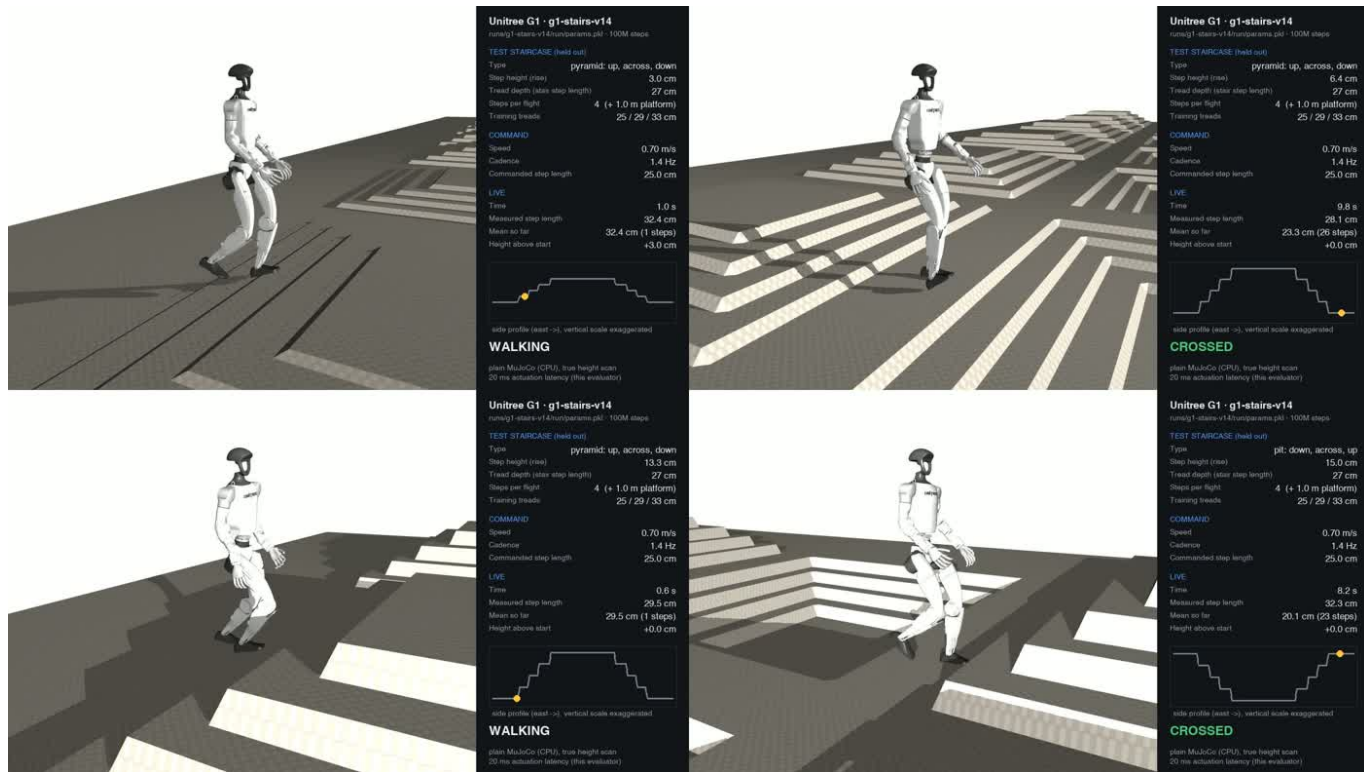


Image : Policy v14 walking over validation staircases of 3.0, 6.4, 13.3 and 15.0 cm in the second physics engine (frames from the simulation video)

Qualification suite: v14 compared with v12

Before a release, every candidate is tested one condition at a time: nominal (strict), head camera, slower walking, a different tread depth, pushes, half friction, a 5 kg payload and extra actuation delay. Training with latency and low friction reduced falls at half friction from 56 % to 6 %, removed all falls under pushes and reduced falls with extra delay from 100 % to 53 %. Motor torque saturation, the share of time that motors are at their limit, dropped from 13 % to 4.6 %, which means lower peak loads on the actuators, gears and bearings.

Table 9: v14 qualification results per test condition

Condition	Runs	Crossed	Fell	Stalled	Qualified height
Strict (true terrain scan)	96	95	1	0	9.9 cm
Strict, head camera	96	86	8	2	6.4 cm
Slower walking, 0.6 m/s	96	89	0	7	11.6 cm
30 cm treads	32	32	0	0	16.0 cm
Pushes (0.3 m/s every 3 s)	32	30	0	2	11.6 cm
Friction ×0.5	32	30	2	0	8.1 cm
5 kg torso payload	32	25	4	3	4.7 cm
Extra delay (40 ms in total)	32	8	17	7	3.0 cm

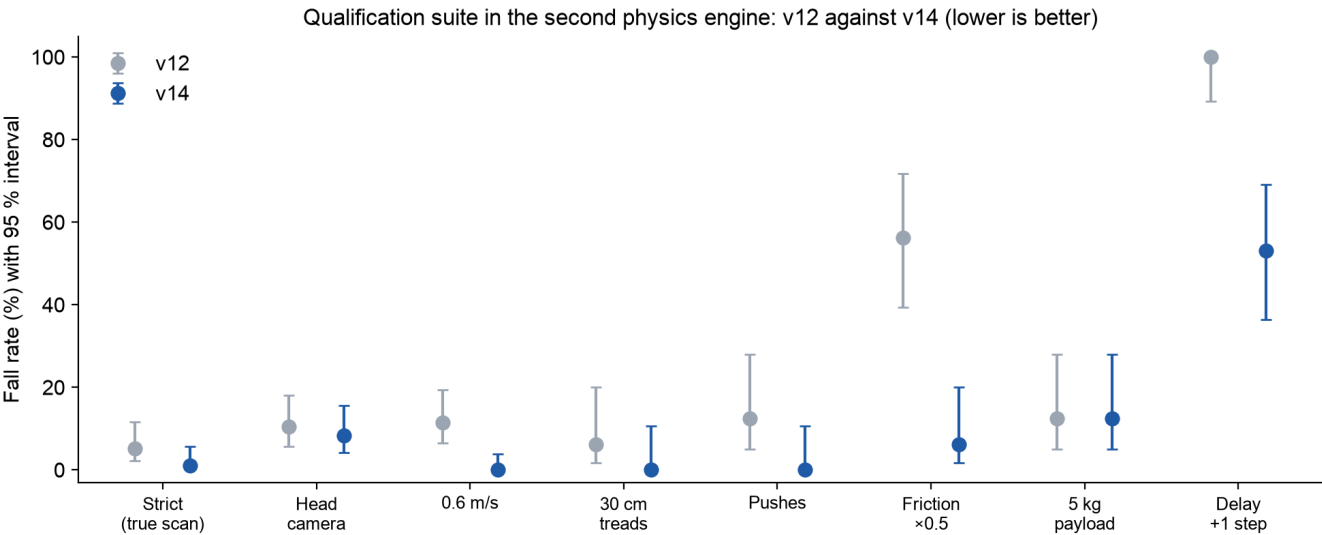


Image : Fall rate per qualification condition with 95 % confidence intervals, v12 compared with v14 (lower is better)

How to read the numbers

A small test can look perfect by luck, so every rate is reported with a 95 % confidence interval, and the release gate checks the pessimistic end of that interval. For example, 1 fall in 96 runs means that the true fall rate could still be as high as 5.7 %; showing a fall rate below 1 % would need about 381 runs without a single fall. During a cross-check we also found that the CPU test program applied one control step (20 ms) of actuation delay in every condition. The "nominal" tests therefore already included 20 ms of latency, and the delay test was really 40 ms. A GPU benchmark with the correct delay reproduced the same results, and the labels in this report are corrected accordingly.

Release decision

With the simulator's true terrain heights, v14 meets the crossing and fall targets. With the head camera, which is the sensor it would actually use, it does not, and it is not yet robust to extra delay or a 5 kg payload. The release gate therefore returned "not passed". In the dashboard (Chapter 9), v14 passes the simulation gate but fails the hardware release gate on its qualified step height (9.86 cm against 10 cm), and the stricter benchmark gate additionally requires the camera, latency and payload conditions. We see this as an important result: the pipeline measured real progress but refused to release a policy on incomplete evidence, which is exactly what an industrial safety process requires. Videos of all versions are available in the [GitHub repository](#).

7.10 Working and Limitations

If it works – why?

The locomotion policy was trained using PPO with several techniques to help it learn the task. The reward function gives feedback on velocity, gait, foot clearance and step length, while penalising behaviours such as falling, slipping and unstable movement. Curriculum learning gradually makes the terrain more difficult, and domain randomisation adds variations in physical parameters, disturbances and sensor noise. We also started the stair-climbing training from an already trained flat-ground walking policy. This gives the robot a basic walking behaviour before learning the more difficult stair task. Based on the simulation results, the policy was able to learn the required locomotion behaviour under the tested conditions.

Limitations:

The current results are limited to the simulation environment and the conditions used during training. The policy may perform differently when transferred to a real humanoid because the simulation cannot perfectly represent real-world effects such as contact dynamics, sensor noise and hardware differences. The policy was also trained for the specific terrain and task conditions used in the experiment, so its performance on different stair dimensions or unexpected obstacles has not yet been fully tested. In addition, successful simulation results do not by themselves guarantee safe or reliable real-world operation.

The measured limitations of the current policy are:

- **Head camera:** 8 of 96 runs fell with the camera. The mean height error is below 1 cm, but errors at step edges (occlusion and smearing, up to 4–7 cm) are exactly where the remaining falls happen.
- **Latency:** with 40 ms of total actuation delay, 17 of 32 runs fell. The real robot's latency must be measured and trained for.
- **Payload:** 4 of 32 runs fell with 5 kg on the torso, because training only varied the torso mass by ± 1 kg.
- **Simulation fidelity:** only the feet collide with the stairs in the simulated model, so a shin or knee hitting a stair edge is not detected. Real stairs are therefore harder than the simulated ones.
- **Privileged information:** in the camera test the controller still receives the exact body velocity and position from the simulator instead of from a state estimator.
- **Validation versus final test:** the validation staircases guided development decisions, so a separate, untouched test set is still needed for an unbiased final result.

How these challenges were handled during the project is described in Chapter 10, and the planned next steps are described in Chapter 12.

8. Implementation of Pick-and-Place: Bearing Ring

To test whether the same pipeline also works for manipulation, the SKF pick-and-place use case (Chapter 11) was implemented in simulation as a new skill: picking a bearing ring from a tray and dropping it onto an inspection fixture.

8.1 Task Definition

The part is the outer ring of a 6206-class deep-groove ball bearing: 62 mm outside diameter, 48 mm bore, 16 mm width and 0.15 kg. The ring must be grasped across its outside diameter so that the fingers never touch the raceway inside the bore, which would contaminate or damage the functional surface. Every episode draws a new tray position, a new fixture position 25–55 cm away and a new ring orientation.

An episode only counts as a success if the ring is lifted, placed within 15 mm of the fixture centre, resting, upright and released. Raceway contact (a finger inside the bore) and dropped rings are counted separately. Each checkpoint is evaluated on 1,024 episodes.

8.2 Robot Arm Stand-in

The first version uses the Franka Panda arm with a parallel gripper from MuJoCo Playground as a stand-in for the G1's arm and hand. This isolates the manipulation problem and verifies the task, the rewards and the evaluation. After 21 million training steps the arm succeeded in all 1,024 episodes (95 % confidence interval 99.6–100 %), with no raceway contact and no dropped rings. A traced episode confirmed a real grasp, lift and placement.

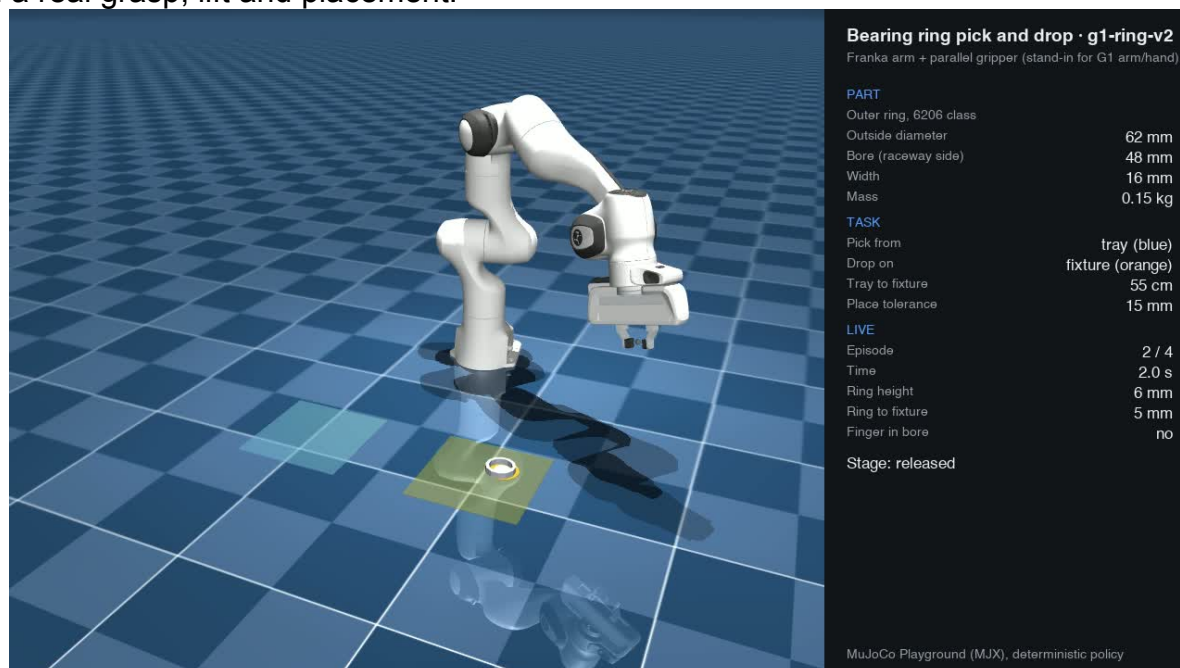


Image : Arm stand-in picking a 6206-class bearing ring from the tray (blue) and placing it on the fixture (orange)

8.3 Unitree G1 with Dexterous Hands

The skill was then moved to the Unitree G1 humanoid with Dex3 dexterous hands at a workbench. The pelvis is fixed, and the policy controls the waist rotation, the right arm and the right hand (15 joints). Version 1 reached 96 % "success", but when an episode was traced, the robot was flicking the ring onto the fixture with its fingers without ever grasping it. The success rule was correct on paper but could be satisfied without a grasp.

Version 2 counts lifting, carrying and success only while the ring is actually held (thumb and finger opposed, and the grasp point within 30 mm of the ring). With this stricter rule the humanoid learned to hold and lift the ring (56 % of episodes at 67 million steps), but it had not yet completed a full placement within 200 million steps. This is an honest intermediate result: a dexterous hand has many more joints than a parallel gripper, and learning a grasp from scratch with reinforcement learning is slow.

Table 10: Bearing-ring pick-and-drop results (1,024 episodes per checkpoint)

Version	Robot	Training steps	Lifted	Success	Raceway contact	Dropped
Arm v2	Franka arm + gripper	21 M	1024	1024	0	0
G1 v1	G1 + Dex3, no grasp check	Final	1015	985 (flick, not valid)	0	0
G1 v2	G1 + Dex3, held grasp required	67 M	575	0	4	3
G1 v2	G1 + Dex3, held grasp required	134 M	357	0	18	51

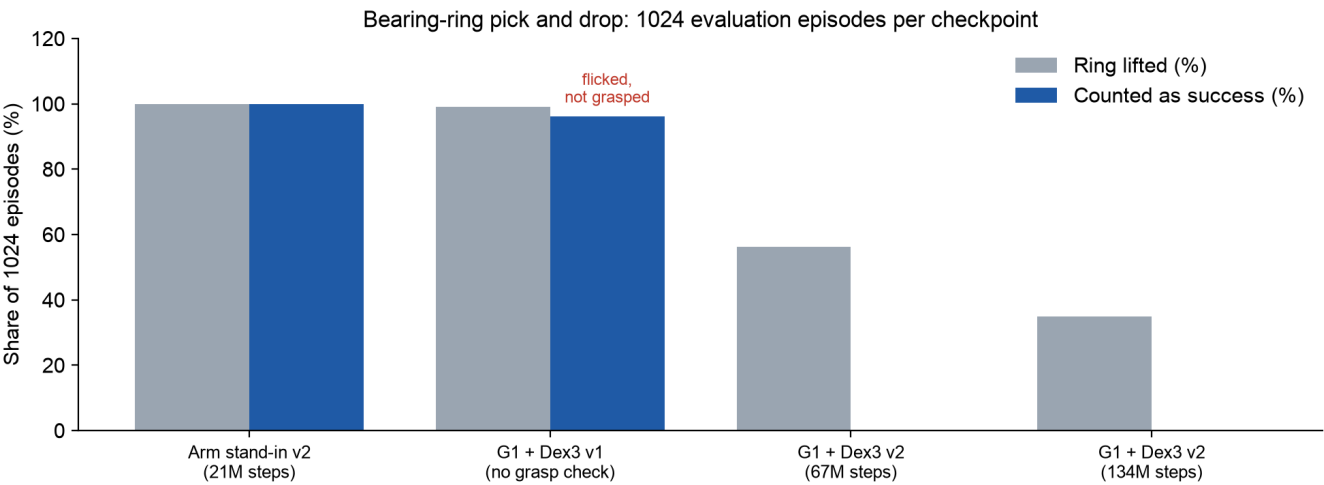


Image : Lift and success rates for the arm stand-in and the G1 humanoid

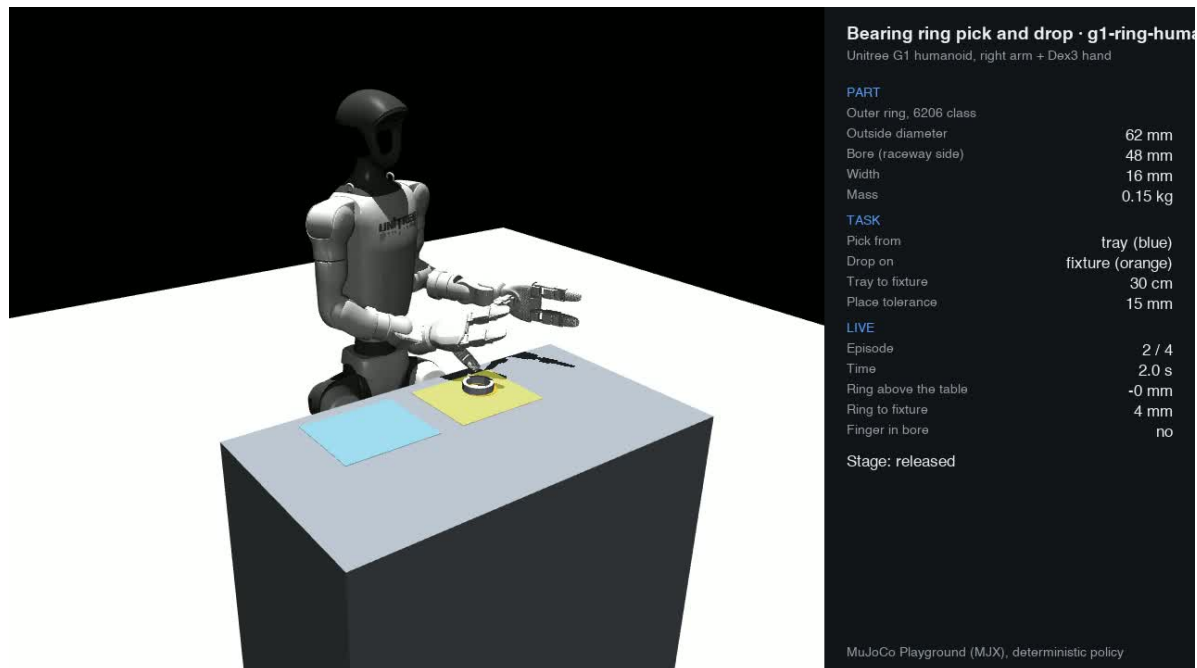


Image : Unitree G1 with Dex3 hand at the workbench: ring on the tray (blue) and inspection fixture (orange)

8.4 Findings and Next Steps

Two lessons carry over to industry. First, a success metric must be verified by looking at individual episodes, not only at the success rate: the 96 % result would otherwise have been a false claim. Second, for a humanoid hand, reinforcement learning alone is not the fastest route. As proposed in Chapter 5, teleoperated demonstrations and imitation learning should provide the initial grasp, and reinforcement learning should then make it robust. The ring skill is already defined in the platform (Chapter 9), so the same evaluation and release gate can be reused when demonstrations are added.

9. SKF Skill Studio: Dashboard, Skills and Skill Composition

A training pipeline that only runs from one engineer's terminal cannot be used by a team. We therefore built SKF Skill Studio, a web platform around the pipeline. A skill is defined once, launched on any compute target, monitored live, evaluated and gated automatically, and approved for release by a person with the right role.

9.1 Dashboard and Source Code

Table 11: Project resources

Resource	Link	Notes
Skill Studio dashboard	https://sylonik.com	Web application. Accounts are created by an administrator (there is no public sign-up); access for the judging panel can be provided on request.
Source code (GitHub)	github.com/lahiruramesh/gtw-hackathon-2026	Training pipeline, evaluation tools, skill manifests, web platform and deployment
Result files	../tree/main/results	Per-episode results and policy cards behind the numbers in this report
Simulation videos	../tree/main/results/videos	Stairs versions v1–v14, step-length demo, bearing-ring pick and drop

9.2 Architecture

The platform consists of a web front end (Next.js), an API (FastAPI), a background worker that manages the life cycle of each run, a PostgreSQL database for skills, runs, metrics and the audit log, Redis for the job queue and S3 storage for checkpoints, videos and results. A small reporter inside each training job streams logs, metrics and heartbeats back to the platform, so progress is visible live in the browser. Jobs can run on the local CPU for quick smoke tests or on cloud GPU machines (such as an NVIDIA L40S) for full training.

The platform is tested with 162 API tests, 33 web unit tests and 16 end-to-end browser tests. Compute credentials are stored encrypted and are write-only, and every change is recorded in an audit log.

9.3 What the Dashboard Can Do

Table 12: Main pages of the Skill Studio dashboard

Page	What it gives the team
Dashboard	Active runs, GPU usage over the last 7 days, items awaiting approval or review, and compute health
Skills	Each skill's pipeline, gate criteria, presets and parameters, and a lineage tree of every run
New run wizard	Choose a skill → a preset or a form → a compute target with a live time estimate → review and launch
Run detail	Live logs and metrics, checkpoints (evaluate one, or warm-start a new run from it), evaluation heat maps per step height, videos, the gate result and the review
Compare	Two to four runs side by side: headline metrics, gate table and overlaid training curves
Methods	The learning-method evaluation matrix of Chapter 2 with adjustable weights
Compute and admin	Compute targets and their health, users, roles and the audit log

The screenshots below are taken from the deployed platform at sylonik.com. All numbers are the real results of the runs described in Chapter 7; nothing was staged for the screenshots.

Recent runs					All runs
Run	Status	Created	GPU-h	Gates	Headline
g1-steplength-aws-e2e-2 Step-length adjustment	● Release gate failed	25 Sept, 23:00	0.9	● Sim failed ● Release failed	Step error — · Grid falls 100%
g1-steplength-aws-e2e-1 Step-length adjustment	● Cancelled	25 Sept, 22:27	0.0	—	Step error — · Grid falls —
g1-stairs-v14 Stair climbing (height scan) Imported	● Release gate failed	25 Sept, 21:02	0.7	● Sim passed ● Release failed	Crossed 99% · Falls 1 · Certified height 9.9 cm
g1-stairs-v12 Stair climbing (height scan) Imported	● Release gate failed	25 Sept, 16:58	1.0	● Sim failed ● Release failed	Crossed — · Falls — · Certified height —
g1-stairs-v11 Stair climbing (height scan) Imported	● Release gate failed	25 Sept, 15:55	1.0	● Sim failed ● Release failed	Crossed 89% · Falls 11 · Certified height 6.4 cm
g1-stairs-v10 Stair climbing (height scan) Imported	● Release gate failed	25 Sept, 13:49	1.2	● Sim failed ● Release failed	Crossed 81% · Falls 18 · Certified height 4.7 cm
g1-stairs-v9 Stair climbing (height scan) Imported	● Release gate failed	25 Sept, 12:20	1.4	● Sim failed ● Release failed	Crossed — · Falls — · Certified height —
g1-steplength-nodr Step-length adjustment · e3-no-dr Imported	● Release gate failed	23 Sept, 19:48	1.6	● Sim failed ● Release failed	Step error 4.4 cm · Grid falls 0%
g1-steplength-v1 Step-length adjustment · v1 Imported	● Release gate failed	23 Sept, 19:41	1.6	● Sim passed ● Release failed	Step error 10.7 cm · Grid falls 0%

Skills					All skills
Skill	Runs	GPU-h total	Best run	Latest gate	
Bearing ring pick and drop	0	0.0	—	—	
Stair climbing (height scan)	5	5.3	g1-stairs-v14	● Failed	
Stair climbing, benchmarked experiments	0	0.0	—	—	
Step-length adjustment	4	4.1	g1-steplength-aws-e2e-2	● Failed	

Image : Dashboard: recent runs with their status, simulation and release gates and headline metrics, and the skills with their best run

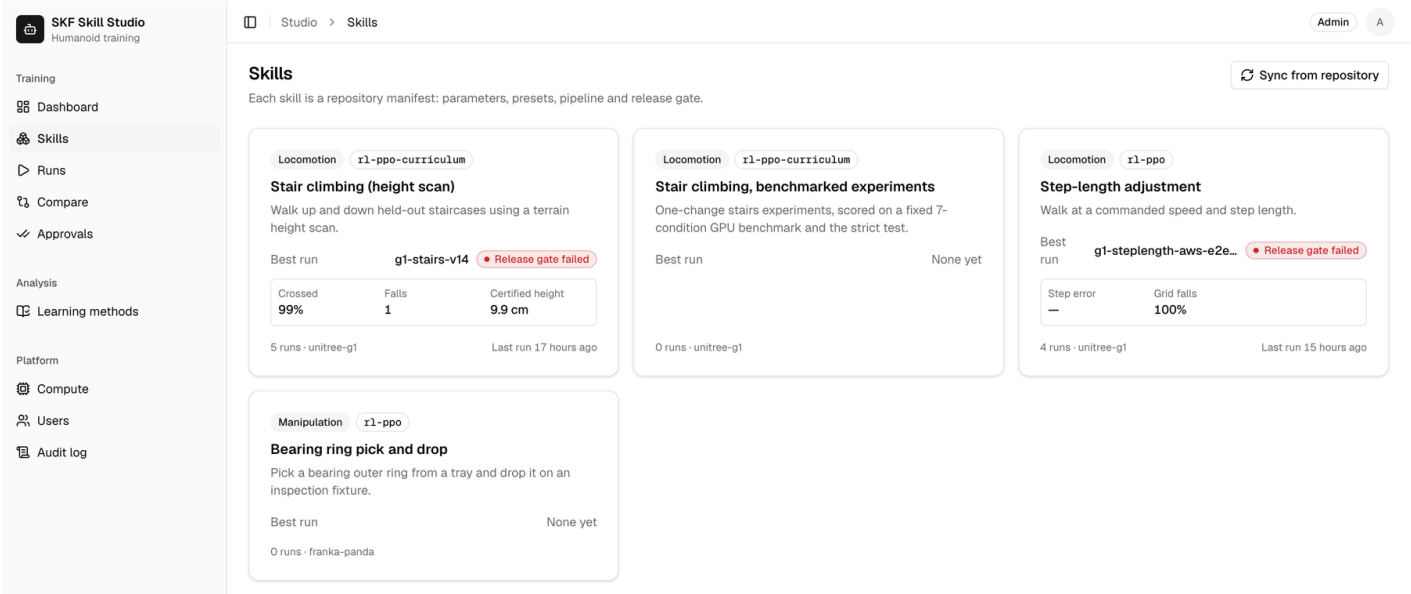


Image : Skills catalogue: each card is one manifest from the repository, with its method, robot and best run

Run detail: pipeline, gates, metrics, evaluation and video

Every run has one page that follows it from launch to release decision. The example below is the best stairs policy, v14. The Gate tab shows the two gate levels described below: v14 passes the simulation gate (99 % crossed, 1 fall) but fails the hardware release gate, because its qualified step height is 9.86 cm against a 10 cm threshold. The Evaluation tab shows the strict test per step height, and the Artifacts tab plays the crossing video recorded by the evaluation stage.

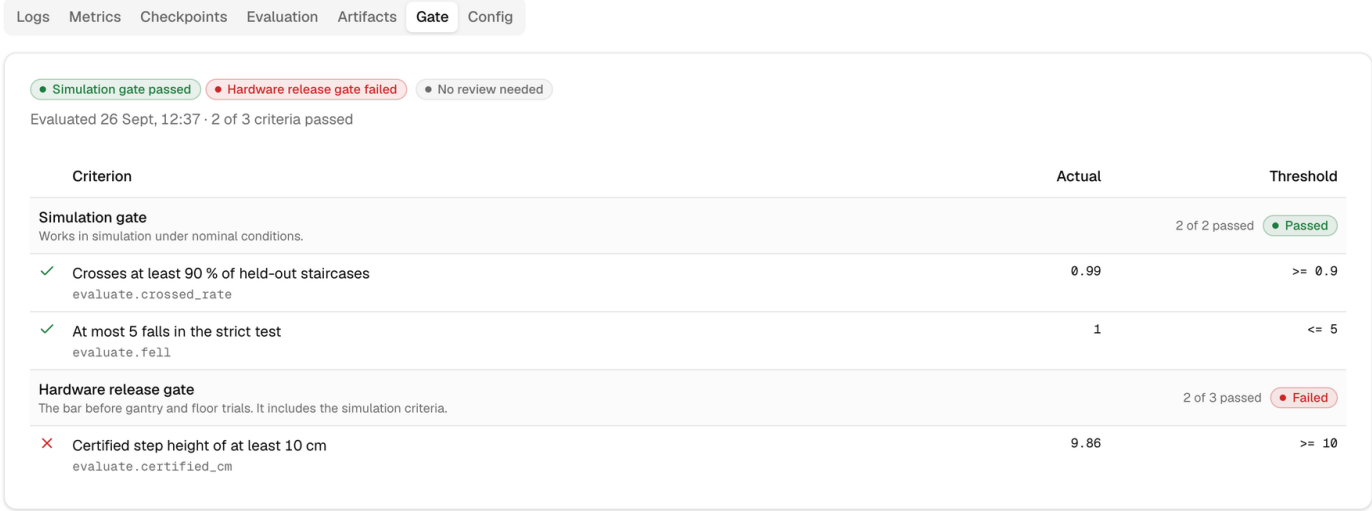


Image : Run detail, Gate tab (v14): simulation gate passed, hardware release gate failed on qualified height (9.86 cm against 10 cm)

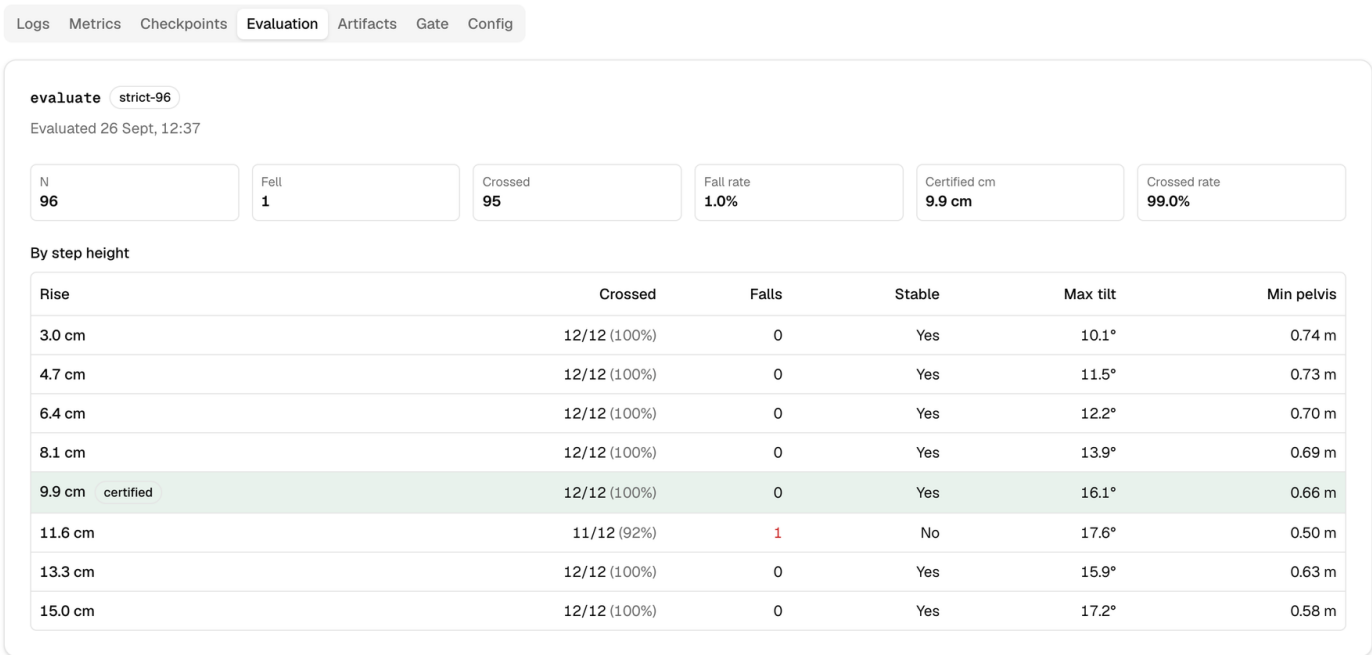
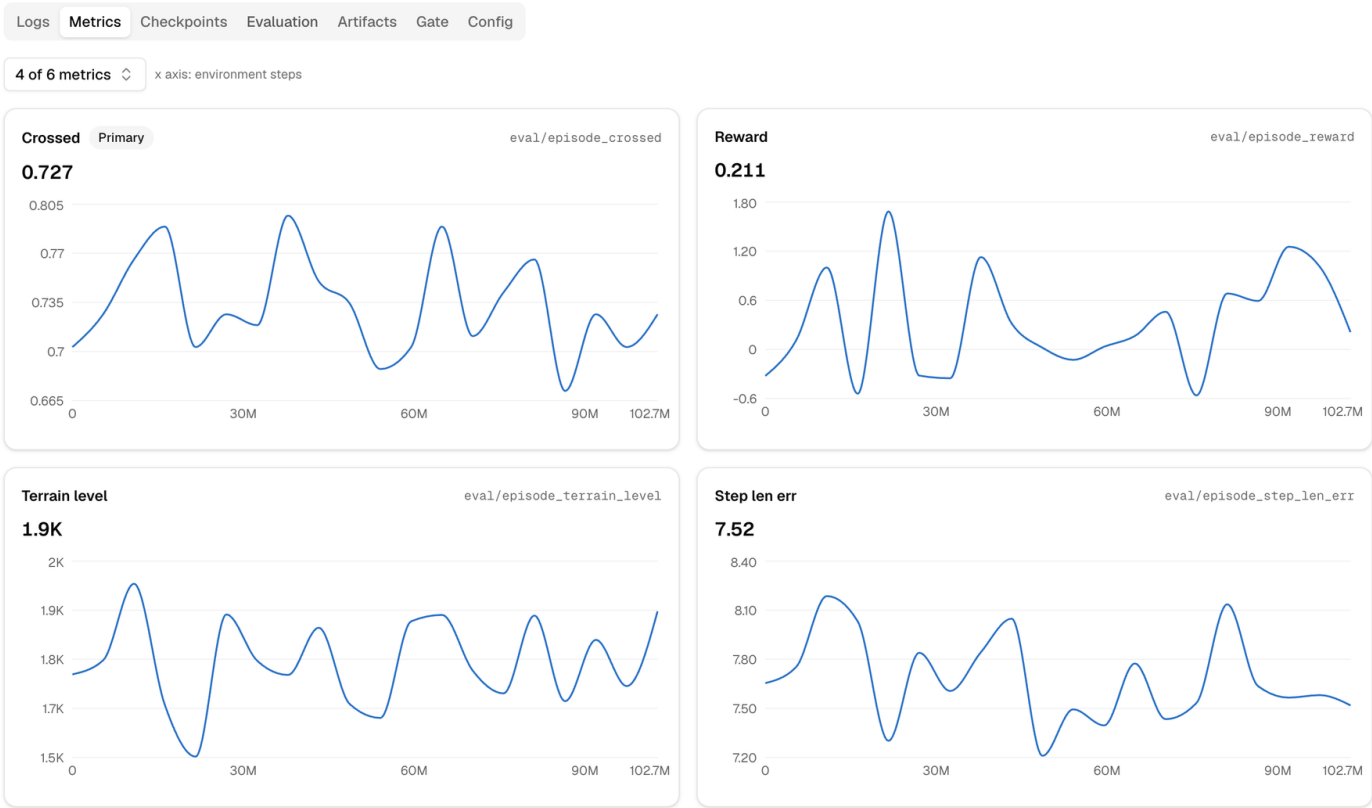


Image : Run detail, Evaluation tab (v14): strict 96-run test per step height, with falls, stability, maximum tilt and minimum pelvis height



Terrain level

eval/episode_terrain_level

1.9K

Step len err

eval/episode_step_len_err

7.52

Image : Run detail, Metrics tab (v14): training curves for crossing rate, reward, terrain level and step-length error

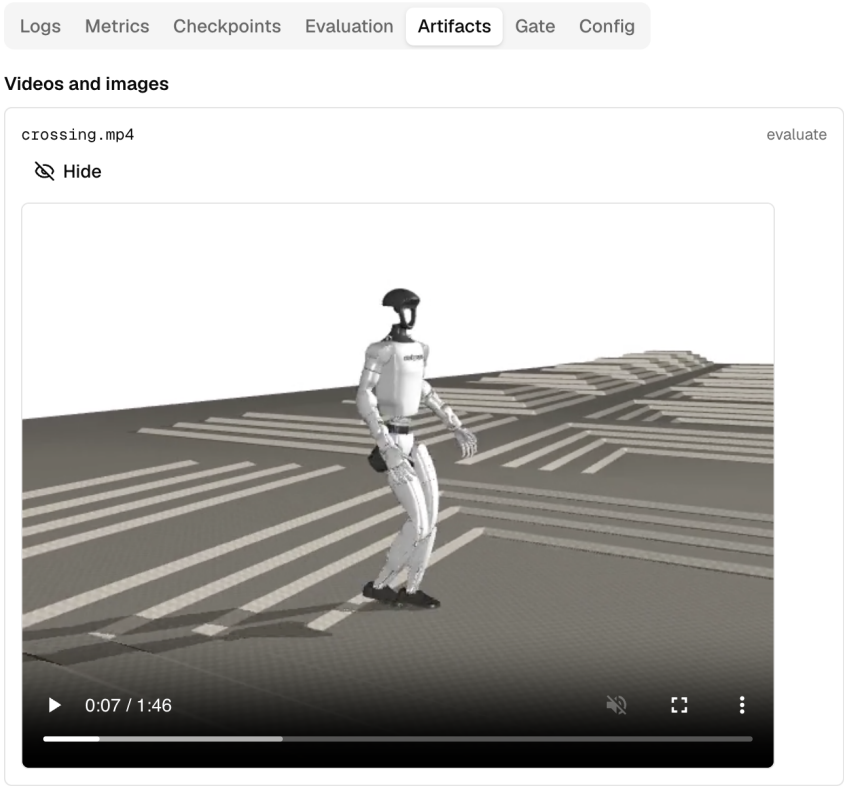


Image : Run detail, Artifacts tab (v14): the crossing video and the stored training files (configuration, progress log and policy weights)

The same gate structure applies to every skill. The step-length policy v1 passes its simulation gate (no falls on the command grid and under 0.5 m/s pushes) but fails the hardware release gate on step-length accuracy in the second engine (10.68 cm against 3 cm) and on 20 ms latency.

Logs

Metrics

Checkpoints

Evaluation

Artifacts

Gate

Config

● Simulation gate passed

● Hardware release gate failed

● No review needed

Evaluated 26 Sept, 12:37 · 2 of 4 criteria passed

Criterion	Actual	Threshold
Simulation gate Works in simulation under nominal conditions. 2 of 2 passed ● Passed		
✓ No falls on the command grid evaluate.grid_fall_rate		
	0	== 0
✓ Survives 0.5 m/s pushes evaluate.stress.push_0.5mps.fall_rate		
	0	== 0
Hardware release gate The bar before gantry and floor trials. It includes the simulation criteria. 2 of 4 passed ● Failed		
✗ Step-length error < 3 cm (unseen engine) evaluate.grid_step_abs_err_cm		
	10.68	< 3
✗ Survives 20 ms latency evaluate.stress.latency_20ms.fall_rate		
	1	== 0

Image : Run detail, Gate tab (step-length v1): two release criteria failed, which the platform reports as a withheld release

Runs are launched from the dashboard as well. The first run launched this way trained the step-length skill on a cloud L40S GPU machine that the platform started automatically, and its logs and training metrics streamed live into the run page while it was running.

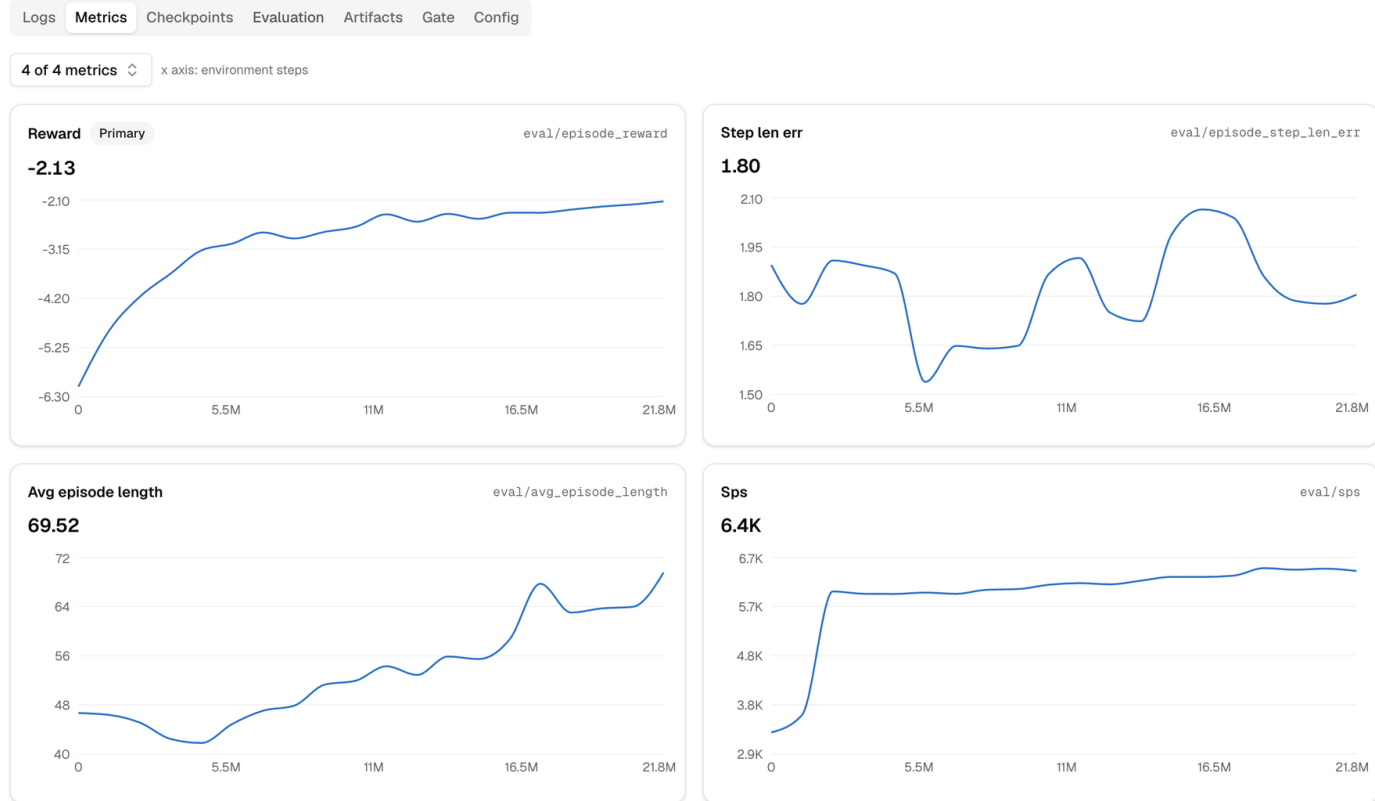


Image : Live metrics of a run launched from the dashboard on a cloud GPU: reward, step-length error, episode length and simulation speed

Compare and Methods pages

The Compare page puts two to four runs side by side, with headline metrics, gate results and overlaid training curves. It makes the progress from v11 to v14 visible at a glance (crossed 89 % → 99 %, falls 11 → 1, qualified height 6.4 → 9.9 cm) and shows the domain-randomisation experiment of Section 7.7 as a direct comparison. The Methods page is the learning-method evaluation of Chapter 2, with adjustable weights and a ranking that updates as the weights change.

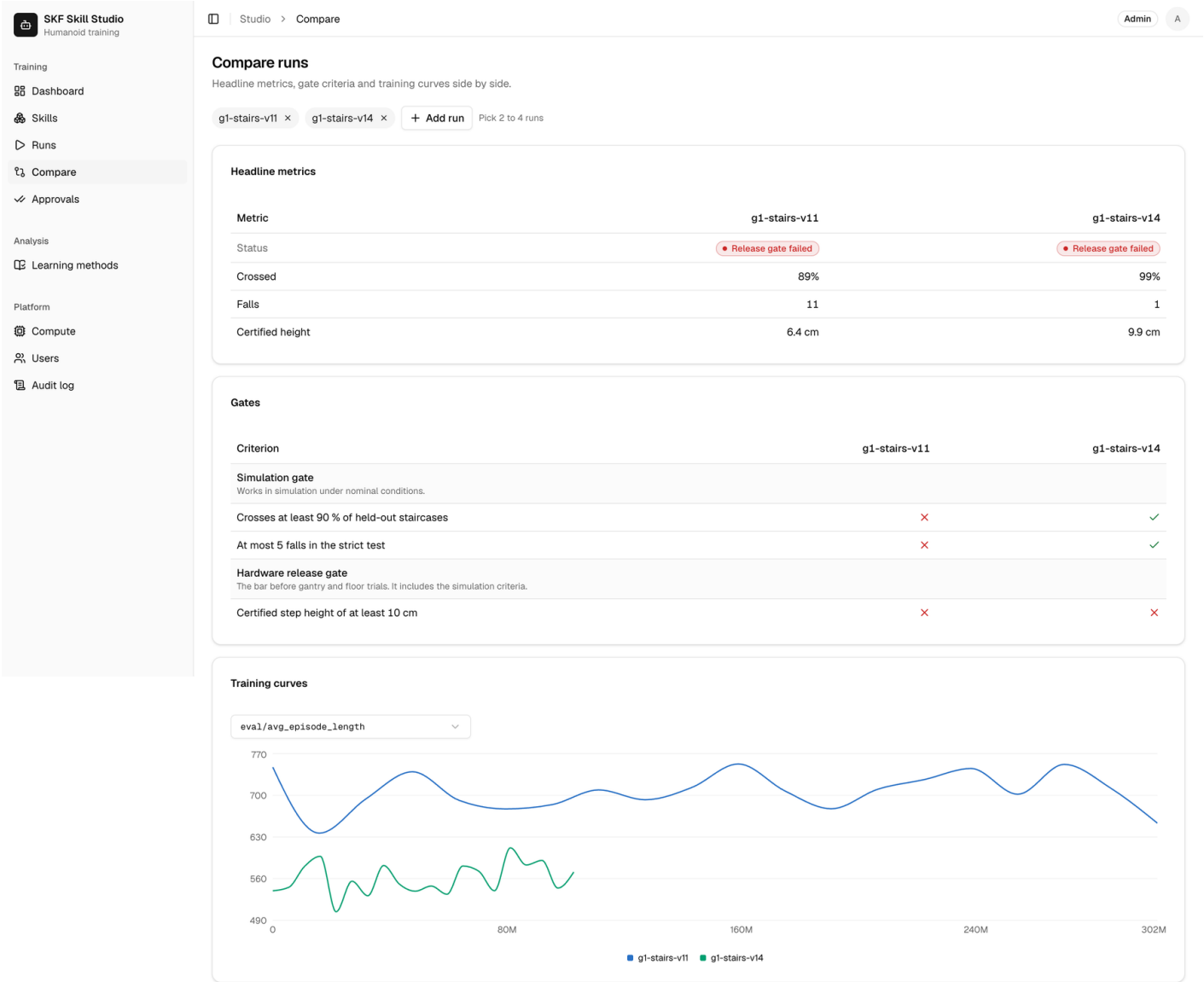


Image : Compare page: stairs v11 against v14, with headline metrics, gate criteria and training curves

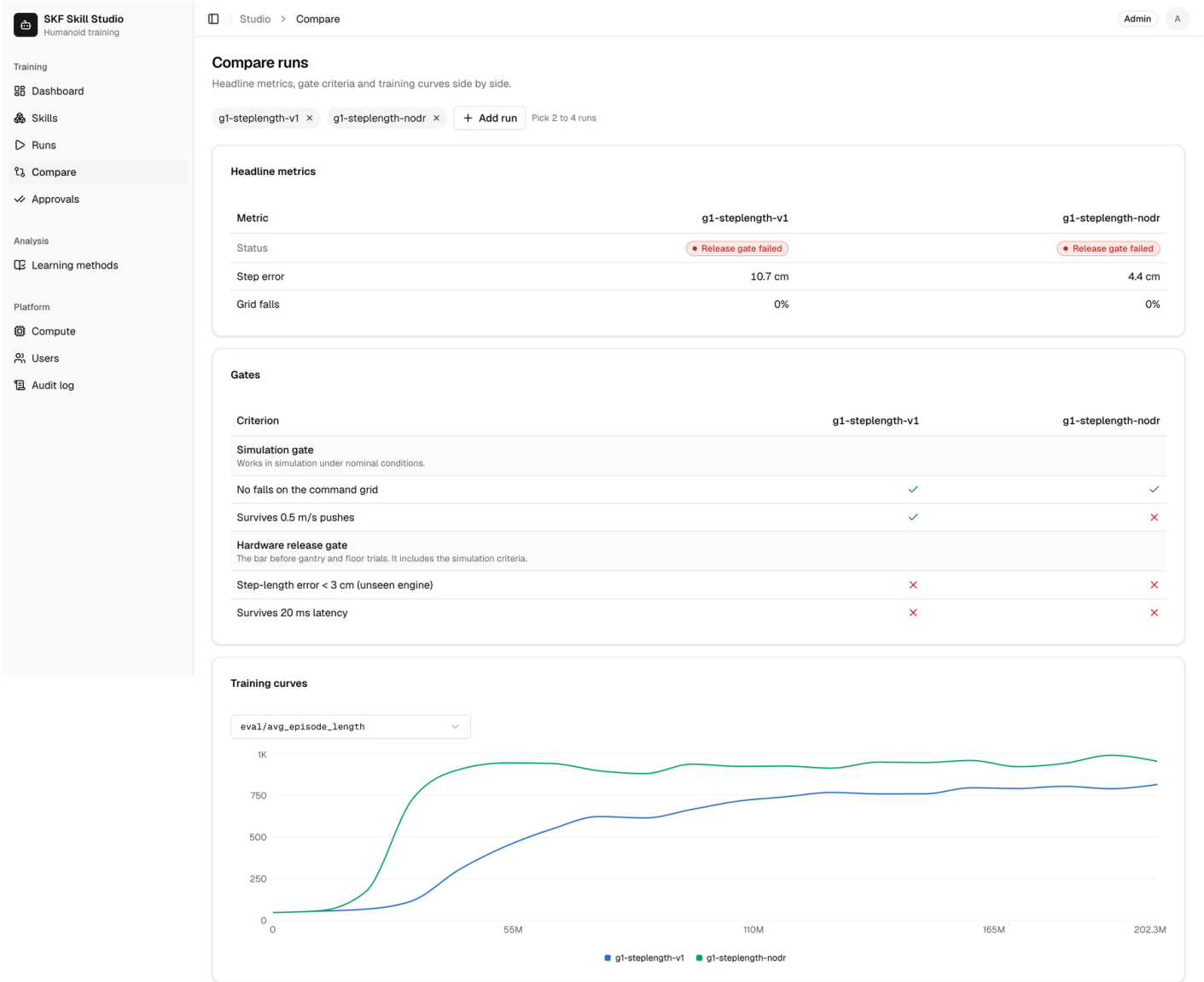


Image : Compare page: step-length policy with domain randomisation (v1) against without (nodr)

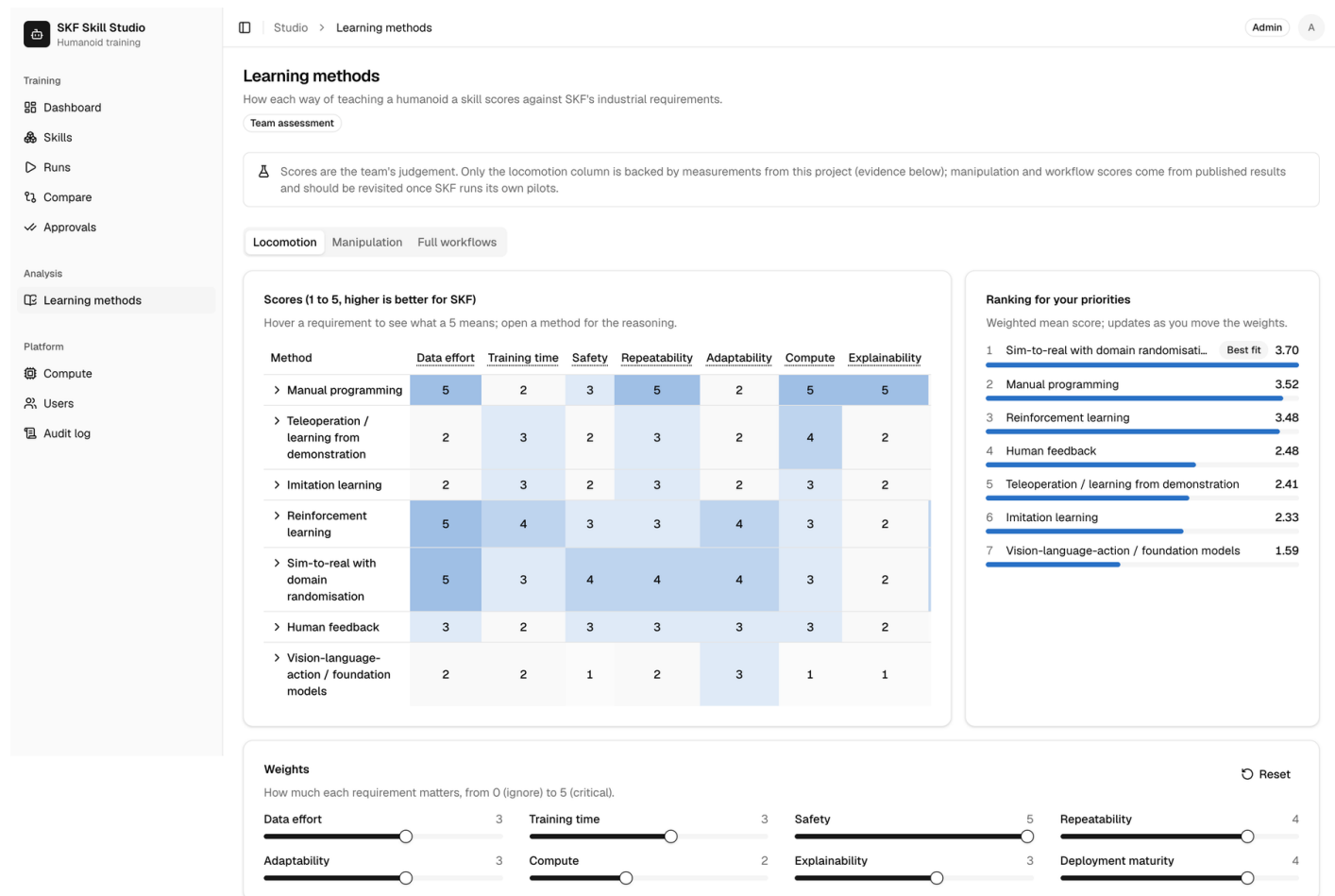


Image : Methods page: the learning-method scores for locomotion, the weighted ranking and the adjustable weights

Two gate levels. Every skill has a **simulation gate**, which shows that the skill works in simulation under nominal conditions, and a stricter **hardware release gate**, which is the bar before gantry and floor trials and includes all the simulation criteria. Only when every criterion passes can a safety reviewer approve a release.

Release governance. A run moves through the states pending approval → queued → running → awaiting review → approved or rejected (or gate failed, failed or cancelled). The gate is evaluated automatically, and a missing metric counts as a failure. The roles are admin, ML engineer, operator (can only launch approved presets), safety reviewer and viewer. Nobody can approve a launch or a release of their own run, and a rejection requires a written reason.

9.4 How to Define a Skill

In Skill Studio a skill is a declarative YAML file, called a manifest, stored in the `skills/` folder of the repository. The platform validates the manifest and turns it into forms, pipelines and gates automatically. A manifest has six parts:

Table 13: Parts of a skill manifest

Part	Purpose	Example (stairs)
Identity	id, name, robot, category and learning method	g1-stairs-bench, unitree-g1, locomotion, rl-ppo-curriculum
params	Typed parameters, shown as a form, with defaults and limits	Training steps, learning rate, leg action scale, warm-start checkpoint
presets	Named, pre-approved parameter sets that operators may launch	Smoke test, v14 control recipe
pipeline	Ordered stages; each runs a command (an argument list, never a shell string) on a chosen target	train → GPU benchmark → strict test → gate
gate	Release criteria as thresholds on metrics from each stage's summary	Fall-rate upper bound $\leq 1\%$, crossed $\geq 90\%$, qualified height $\geq 10\text{ cm}$
headline	The metrics shown on run cards and in comparisons	Gap to targets, falls with the camera, qualified height

The dashboard renders each manifest as a skill page. For the stairs skill it shows the pipeline stages, the two gate levels with their thresholds and the typed parameters with their defaults and allowed ranges, all generated from the YAML file:

SKF Skill Studio
Humanoid training

Training

Dashboard

Skills

Runs

Compare

Approvals

Analysis

Learning methods

Platform

Compute

Users

Audit log

Studio > Skills > Stair climbing (height scan)

AdminA

Stair climbing (height scan)

Walk up and down held-out staircases using a terrain height scan.

Locomotionrl-ppo-curriculumActiveunitree-g1 · manifest at 6024ef8

OverviewPresetsRunsLineage

Pipeline

Every run executes these stages in order.

Train (Brax PPO, stairs curriculum)
train · Runs on the chosen compute target

>

Strict test (96 crossings, held-out stairs)
evaluate · Runs on the worker (CPU)

>

Simulation and release gates
gate · Runs on the chosen compute target

Description

The G1 walks straight over pyramids (up, across, down) and pits (down, across, up) with step heights it never saw in training. The policy observes a height scan around its feet; with `scan_model: camera` it is trained with the errors of the simulated head depth camera.

Warm-started from the flat step-length policy and refined over 11 versions (curriculum, sole contacts, leg action range, stairs cadence). The **strict test** runs every staircase of the held-out layout from 3 start variants (96 crossings) at 0.7 m/s. A step height is **certified** when it and every lower height pass with no falls, torso tilt < 25° and pelvis > 0.55 m.

Gates

All criteria must pass before a safety reviewer can approve a release.

Criterion	Threshold
Simulation gate Works in simulation under nominal conditions.	
Crosses at least 90 % of held-out staircases <code>evaluate.crossed_rate</code>	>= 0.9
At most 5 falls in the strict test <code>evaluate.fell</code>	<= 5
Hardware release gate The bar before gantry and floor trials. It includes the simulation criteria.	
Certified step height of at least 10 cm <code>evaluate.certified_cm</code>	>= 10

Parameters

Parameter	Type	Default	Allowed
Training steps <code>timesteps</code>	integer	200,000,000	20,000 to 2,000,000,000
Smoke test (tiny CPU run) <code>smoke</code>	yes / no	No	—
Height-scan noise model <code>scan_model</code>	choice	uniform	uniform, camera
Leg action scale (rad) <code>leg_action_scale</code> Target range of hip pitch, knee and ankle pitch; empty keeps Playground's 0.5	number, optional	—	0.1 to 2
Learning rate <code>lr</code> PPO learning rate (Playground G1: 3e-4; lower to fine-tune)	number, optional	—	0.000001 to 0.01
Seed <code>seed</code>	integer	0	≥ 0
Warm start from <code>init_from</code>	checkpoint, optional	—	—

Image : Skill page for stair climbing, generated from its manifest: pipeline, description, gates and parameters

A shortened example of the stairs manifest:

```
id: g1-stairs-bench
name: Stair climbing, benchmarked experiments
robot: unitree-g1
category: locomotion
method: rl-ppo-curriculum
params:
  timesteps: {type: integer, default: 150000000}
  lr:        {type: number,  default: 0.0001}
  init_from: {type: checkpoint, nullable: true}      # warm start
presets:
```

```

- {id: smoke, name: Smoke test, params: {smoke: true, timesteps: 20000}}
pipeline:
- id: train
  kind: train
  runs_on: target                                # the selected GPU
  argv: [python, -m, glpipe.train, --task, stairs, --out, "{out_dir}"]
  flags: {lr: --lr, init_from: --init-from}
- id: bench
  kind: evaluate
  runs_on: target                                # 7 conditions x 2048 episodes
  argv: [python, -m, glpipe.bench, "{input_dir}/train/params.pkl"]
- id: evaluate
  kind: evaluate
  runs_on: local                                # strict test, second engine
  argv: [python, -m, glpipe.stairs_eval, "{input_dir}/train/params.pkl", --starts, "3"]
- id: gate
  kind: gate
gate:
- {metric: bench.camera.rate_hi, op: "<=", value: 0.02}
- {metric: evaluate.crossed_rate, op: ">=", value: 0.9}
- {metric: evaluate.certified_cm, op: ">=", value: 10}

```

Defining a new skill follows five steps:

- **Step 1 – Specify the task and success criteria:** decide what the skill must do and write the gate thresholds first, before any training.
- **Step 2 – Choose parameters and presets:** expose only the settings that should be changed, with safe limits, and create presets for operators.
- **Step 3 – Describe the pipeline:** list the train, evaluate and gate stages and the command that each stage runs.
- **Step 4 – Add a summariser:** a short Python script (`summarize.py`) turns each stage's output into the metrics that the gate reads.
- **Step 5 – Commit the file:** the platform validates the manifest, and the skill appears in the dashboard ready to launch.

For a supported task type, a new variant (for example another gate, other presets or a different benchmark) is simply a new file. A genuinely new task still needs a simulation environment and an evaluator: the ring pick-and-drop skill required about 450 lines of environment and evaluation code before its 64-line manifest could run. Four skills are defined today:

Table 14: Skills defined in the platform

Skill	Stages	Release gate
g1-step-length	train → evaluation suite → gate	No falls on the command grid; step error < 3 cm; no falls with 0.5 m/s pushes or 20 ms latency
g1-stairs	train → strict 96-run test → gate	Crossed ≥ 90 %; at most 5 falls; qualified height ≥ 10 cm
g1-stairs-bench	train → 7-condition GPU benchmark → strict test → gate	10 criteria, including camera, delay, friction, payload and pushes
ring-pick-drop	train → 1,024 episodes → gate	Success ≥ 99.5 % (lower bound); no raceway contact; no dropped ring

9.5 How to Combine Skills

Skills become more valuable when they are combined. The platform supports combination at training time and at run time, as the following four examples show.

Example 1 – Chaining skills through warm starts

A new skill can start from a released checkpoint of another skill by setting `init_from`. The stairs skill starts from the flat-walking step-length skill, and each stairs version starts from the best previous one. The dashboard records this lineage as a tree, so it is always clear which checkpoint a policy came from. In practice this reduced a new iteration from days of training from scratch to about one hour.

```
# launch parameters: a stairs run warm-started from the step-length skill
skill: g1-stairs
preset: full
params:
  init_from: g1-step-length / run v1 / params.pkl
  timesteps: 300000000
```

Example 2 – Variants of the same task

g1-stairs and g1-stairs-bench train the same task but use different gates. g1-stairs-bench runs one-change experiments: a control run with the best recipe plus "arms" that each change one setting (for example the share of episodes with actuation delay, or a 0–4 kg payload). All runs are warm-started from the same checkpoint and compared on the same fixed benchmark, which makes it possible to tell which change really helped.

Example 3 – Layering skills at run time

On the robot, skills run in layers at different speeds. The learned 50 Hz gait policy keeps balance, a classical controller holds the heading, safety code enforces limits (for example, zero speed is commanded above 25° of tilt) and an optional supervisor agent reads the operator's mission and chooses the gait mode at 2 Hz. In a stairs test world the supervisor learned from its own outcomes that the blind vendor policy can cross 4 cm steps but not 8 cm steps, and it now stops in front of 8 cm stairs even when told to climb. All nine regression runs gave the expected outcome without a fall.

Example 4 – Composing an SKF workflow from released skills

The SKF use case combines locomotion, manipulation and a workflow: a humanoid carries bearing rings from a washing line up a staircase to an inspection station, places them and wipes the inspection surface. We propose to build such jobs as a state machine that only calls released skills, each with its own gate:

```
workflow: mezzanine-ring-transfer          # proposed
steps:
  - walk_to: {skill: gl-step-length, target: washing_line}
  - pick:    {skill: ring-pick-drop, from: tray, to: carrier}
  - climb:   {skill: gl-stairs, payload_kg: 3}
  - place:   {skill: ring-pick-drop, from: carrier, to: fixture}
  - wipe:    {skill: surface-wipe}          # future skill
on_failure: stop_and_call_operator
requires: every skill released (gate passed + safety review)
```

Because each step is a separately qualified skill, a failure can be traced to one skill, and that skill can be improved and re-released without retraining the whole job. Workflow composition is designed but not yet implemented in the platform.

10. Challenges, Techniques and Lessons Learned

Most of the effort in this project went into problems around the learning algorithm rather than into the algorithm itself. This chapter summarises the main challenges, how we overcame them and the techniques that made the difference. Each solution was turned into an automatic check in the pipeline, so the same problem cannot silently happen again.

10.1 Challenges and How We Overcame Them

Table 15: Main challenges and how they were overcome

Challenge	What we observed	How we overcame it
Unrealistic foot contact	The GPU simulator used one contact point per foot and the test engine 3–20; eight versions trained on the wrong contact	Six contact spheres per sole; a preflight check compares both engines before every run
Robot too weak on high steps	The ± 0.5 rad knee target range capped knee torque near 75 Nm (the motor gives 139 Nm); the pelvis collapsed on steps of 8 cm and more	Leg action range widened to 1.0 rad; the failure report shows the time spent at joint limits
Reward loophole	Version 3 learned to turn away from difficult stairs	Turning more than 60° off the heading ends the episode as a fall; heading deviation is checked in every evaluation
Over-fitting	Version 5 qualified 8 cm on its own stairs but only 0–4.7 cm on unseen stairs	Mixed tread depths in training; testing only on validation staircases
Small tests measure noise	Changing the step command by 4 mm moved a 32-run test from 30/32 to 27/32	2,048-episode GPU screening per checkpoint; 96-run strict test with 3 start poses; confidence intervals on every rate
The last checkpoint is not the best	In v10 and v11 the final checkpoint was worse than an earlier one	Every checkpoint is ranked, and the best one is qualified
Sensitivity to latency	Every policy fell under extra actuation delay	Random actuation delay in training (falls with extra delay 100 % $\rightarrow$ 53 %); an evaluator latency offset was found and disclosed
Slow training	The GPU physics engine printed a warning at every step: training ran 3–5 $\times$ slower and produced 30 GB of logs	A throughput check in the preflight; the warning was disabled, giving about 5 $\times$ faster training
Compute availability	GPU machines were not always available in one zone, and long runs risked running unattended	A one-command job that finds a GPU in any zone, sets a hard shutdown on the machine and stops it when training ends
Camera perception gap	Falls with the camera 8/96 against 1/96 with the true terrain; fine-tuning on the camera model did not help	Camera error model in training; teacher–student distillation planned
False success in manipulation	The humanoid "succeeded" in 96 % of episodes by flicking the ring without grasping it	Success is only counted while the ring is held; every new success rate is verified by tracing an episode
One-person workflow	Results depended on one engineer's terminal and memory	Skill Studio: manifests, automatic gates, live monitoring, lineage and role-based approval

10.2 Key Techniques

- **Massively parallel reinforcement learning:** PPO with 8,192 simulated robots on one GPU, about 100,000 steps per second.
- **Terrain curriculum and hard-case replay:** every robot trains at the edge of its ability, with extra practice on the steps where falls happen.

- **Warm starts (transfer learning):** each skill and version starts from the closest working policy, with a check that the starting behaviour is identical.
- **Targeted domain randomisation:** friction, masses, pushes, sensor noise, actuation delay and a camera error model, chosen to match real site conditions.
- **Asymmetric actor-critic:** the robot policy uses realistic sensors, while the critic uses privileged simulator information during training only.
- **Two-engine testing:** train in MuJoCo Warp/MJX and qualify in MuJoCo C, which catches policies that exploit one simulator.
- **Statistical evaluation:** thousands of episodes per checkpoint, confidence intervals, separate labels for falls and stalls, and gates that use the pessimistic bound.
- **Release gate with human review:** thresholds decide automatically, and a safety reviewer who did not train the policy approves the release.
- **Hybrid layering:** a learned gait, classical heading control, safety code and an optional supervisor agent, each doing what it is best at.

10.3 What We Learned

The largest improvements followed changes that made the simulator and the robot's action range physically right, while reward tuning mostly closed loopholes. For an industrial team this suggests that the scarce skill is physics and evaluation engineering rather than machine-learning research, and that good tooling can capture much of this knowledge. Our answer to the question of how to make humanoid training easier and more efficient is: make every lesson a check, make every check automatic, and let people approve what the checks cannot see.

11. SKF Use Case and Business Value

11.1 SKF use case: Pick-and-Place

Pick-and-place can be used in industrial environments for tasks such as “component handling, machine loading and unloading, and moving parts between workstations”. For example, a humanoid robot could pick a bearing component and place it into a machine, move components from a storage area to a workstation, or transfer parts between different stages of a production process. These tasks require capabilities such as object detection, grasping, movement, and accurate placement.

The flowchart below presents the overall training and validation process for teaching a humanoid robot to perform the pick-and-place task. The process starts with defining the task and success criteria, followed by demonstrations, policy training, simulation testing, safety validation, and transfer to the physical robot.

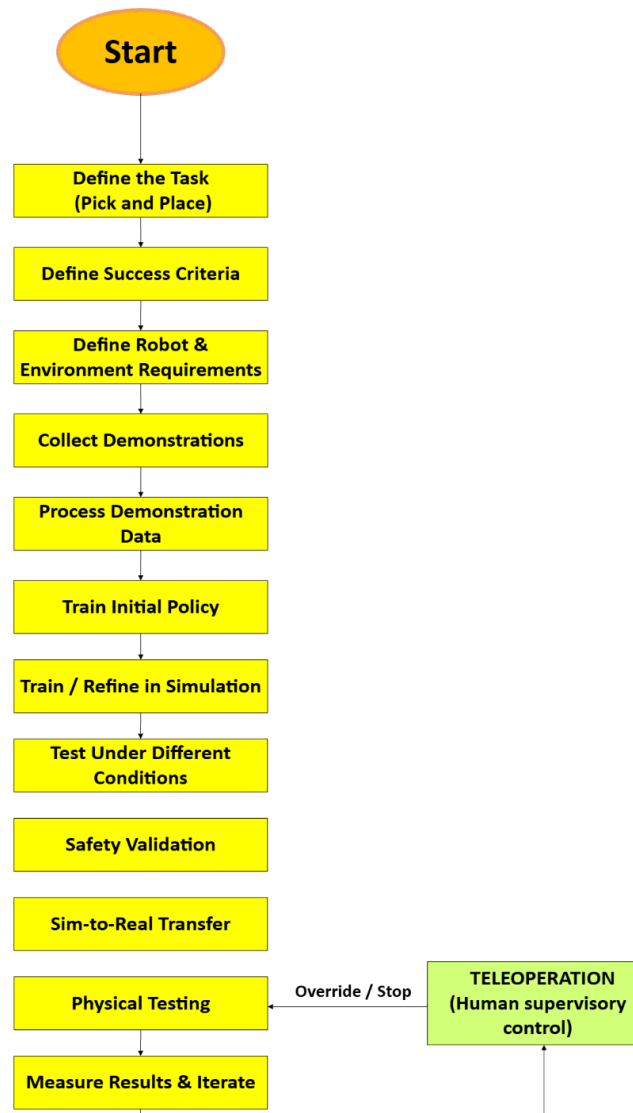


Image : Flow chart for SKF Use case: Pick-and-Place

Task List for Pick-and-Place:

- Define the Task: Specify what the humanoid must pick, where it must move the object, and where it must place it.
- Define Success Criteria: Define measurable outcomes such as successful grasp, correct placement, no collision, and task completion time.
- Define Robot & Environment Requirements: Define the humanoid, hand, workspace, sensors, and simulation environment.
- Prepare Object & Target: Define the object properties, starting position, and target location.
- Collect Demonstrations: Provide examples of the desired manipulation behaviour through human demonstrations or teleoperation.
- Process Demonstration Data: Convert the demonstrations into suitable robot states, actions, and trajectories.
- Train Initial Policy: Use the demonstrations to learn an initial manipulation behaviour.
- Train / Refine in Simulation: Improve the policy through repeated simulated trials and, where appropriate, reinforcement learning.
- Test Under Different Conditions: Change object position, orientation, and other conditions to test adaptability.
- Safety Validation: Check for collisions, unstable movements, failed grasps, and other unsafe behaviour.
- Sim-to-Real Transfer: Transfer the validated policy from simulation to the physical humanoid.
- Physical Testing: Perform the pick-and-place task under controlled real-world conditions.
- Measure Results & Iterate: Compare the results against the success criteria and improve the pipeline if required.
- Teleoperation Override / Stop: If a problem occurs during physical testing, the operator can intervene or stop the robot.

11.2 Locomotion vs. Manipulation and Business KPIs

To assess the industrial relevance of humanoid robotics for SKF, locomotion and manipulation are compared across key business and technical KPIs. The comparison highlights their respective benefits, limitations, safety requirements, data needs, industrial readiness, and potential SKF use cases.

Table 16: Business KPI Assessment – Locomotion vs. Manipulation

Business KPI's	Locomotion	Manipulation
Benefits	Flexible movement between production areas; access to existing workstations	Repetitive-task automation; flexible part handling; machine tending
limitations	Balance; floor/terrain constraints; fall risk; limited direct production value	Grasping; precision; cycle time; process variation
Safety	Fall prevention; worker interaction; safe navigation around machinery	Pinch/crush hazards; dropped parts; human–robot interaction
Industrial readiness	Developing	More mature
Data needs	Factory layout; walking paths; floor conditions; obstacles; sensor data	Part geometry; demonstrations; camera data; grasp/contact data; process variations
External partners	Humanoid supplier; simulation provider; safety/integration partner	Robot supplier; vision provider; gripper/tooling supplier; system integrator
Active use of AI	Perception; navigation; adaptive locomotion; obstacle detection	Vision; grasp planning; task learning; anomaly detection
Control systems	Navigation; balance control; whole-body control; safety systems	Motion control; force control; gripper control; PLC/machine integration
Potential SKF Use Cases	Moving between workstations; component transport; accessing different production areas	Pick & put; machine loading/unloading; component handling; inspection; wiping/cleaning

11.3 Business Value for SKF

Humanoid robotics represents a potential extension of SKF’s existing expertise in bearings, precision components, and industrial applications. Humanoid robots contain numerous joints and rotating interfaces that require high-precision and reliable transmission components, creating a direct connection to SKF’s core capabilities. This opportunity is reinforced by SKF’s 2026 agreement with Chinese robotics-component manufacturer Leaderdrive to establish a venture focused on high-precision transmission components for humanoid robot joints. SKF will hold a 60% majority stake in the venture, which is also intended to serve selected international markets, including Europe.

For SKF, this could create an opportunity to expand beyond component supply and develop a broader position in the humanoid robotics value chain. One potential approach would be to combine hardware sourcing and manufacturing capabilities in China with software, integration, validation, and deployment capabilities in Sweden or elsewhere in Europe. This could allow SKF to establish a European industrial robotics offering while maintaining access to the rapidly developing Chinese robotics supply chain.

At the same time, European deployment of connected and AI-enabled robots introduces considerations related to cybersecurity, data governance, software control, and regulatory compliance. The EU Cyber Resilience Act establishes cybersecurity requirements for products with digital elements, while the EU AI Act introduces additional requirements for certain AI systems and applications.

This creates a potential strategic model in which SKF combines its established expertise in bearings and industrial manufacturing with robotics hardware, AI-based control, and European system integration. Such a model could allow SKF to participate in the emerging humanoid-robot market while maintaining greater control over software, data, cybersecurity, and customer-specific integration within the European market.

11.4 Measured Results Relevant to SKF

The project produced measurable results that indicate how a humanoid skill programme could run in practice:

Table 17: Measured pipeline results

Measure	Result
Time per training iteration	2.6 hours → 41 minutes from launch to ranked checkpoints
Fall rate on unseen staircases (true terrain scan)	87.5 % (v1) → 1.0 % (v14)
Motor torque saturation during stair climbing	13 % → 4.6 % of the time
Evaluation scale per checkpoint	32 → 2,048 crossings
Development time to 9.9 cm stairs (true terrain scan)	3 days, 14 versions
Bearing-ring pick and drop, arm stand-in	1,024 of 1,024 episodes successful, no raceway contact
Coarse step-length control	Available without any training (baseline sweep)

The lower torque saturation is directly relevant to SKF. A policy that keeps motors and joints away from their limits reduces peak loads on actuators, gears and bearings, which affects component life. In future, the policy cards produced by the pipeline could include joint load profiles for every released skill, as an input to bearing selection, lifetime estimation and condition monitoring in humanoid joints.

12. Future Work

Future work can focus on further research and simulation of locomotion and manipulation under realistic industrial conditions. More extensive simulation and testing can help produce stronger results and identify potential safety and performance issues before deploying the system on physical humanoid robots. These results can then support a controlled transition towards real-world testing and eventual industrial deployment.

12.1 Next Technical Steps

- Fix the evaluator latency offset and re-run the qualification of v12 and v14.
- Add full leg–terrain collisions and realistic stair nosings, so that simulated stairs are as hard as real ones.
- Put a state estimator in the loop instead of the exact simulator velocity and pose, and test calibration errors, delays and map drift.
- Measure the real G1's latency and the floor friction at the site, and use them as randomisation ranges.
- Run the one-change experiments that are already defined (delay share, 0–40 ms delay, 0–4 kg payload).
- Use teacher–student distillation to train a camera-based student from the privileged-scan policy and close the camera gap.
- Add teleoperated demonstrations and imitation learning to the ring skill, and a wiping skill with force control.
- Freeze the best policy and evaluate it once on a new, untouched test set of staircases.
- Move to hardware step by step: hardware in the loop, a harnessed robot on a gantry, supervised trials and shadow operation.

12.2 Agents to Review the Process

Today, many review steps depend on an engineer noticing a problem. The one-point foot contact, the reward loophole, the ring flick and the latency offset were all found by a person looking closely at the data. The next step is to add AI review agents to every stage of the pipeline. The agents read logs, metrics, per-episode results, videos and code, explain what they find and propose actions. They do not approve anything: the deterministic gate thresholds decide pass or fail, and a human safety reviewer approves every release.

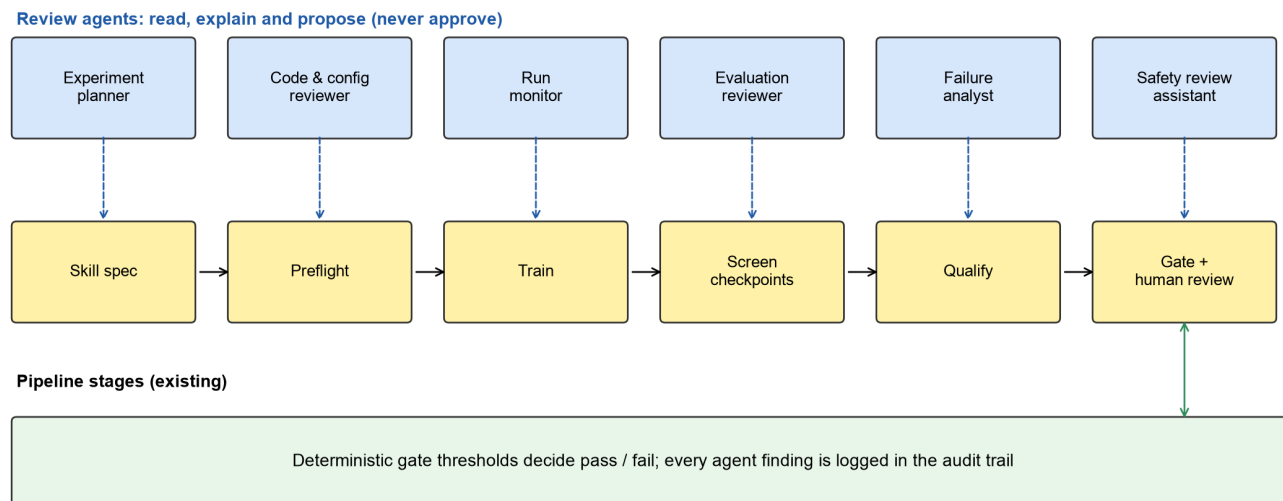


Image : Proposed review agents attached to each stage of the training pipeline

Table 18: Proposed review agents

Agent	Stage	What it reviews	Example from this project
Experiment planner	Before training	Proposes the next one-change experiment from past results, within a whitelist of allowed settings	Proposed the delay-share and payload experiments after v14 (built as the improvement-loop advisor)
Code and configuration reviewer	Before training	Reviews changes to environment, reward and evaluation code; checks that the pre-flight and parity tests pass	Would flag an off-by-one error such as the evaluator's action queue
Run monitor	During training	Watches logs and metrics live; detects stalls, reward collapse, throughput drops and log floods; can pause a run	Would have stopped the 30 GB log flood within minutes
Evaluation reviewer	After screening	Traces sample episodes and videos to confirm that successes are real; checks sample sizes and repeated comparisons	Would have caught the ring "flick" before it was reported
Failure analyst	After qualification	Groups failures by condition and step height, links them to known root causes and proposes new test conditions	Camera falls clustered at 9.9 cm and 13.3 cm
Safety review assistant	Before release	Prepares the policy card and a checklist for the human safety reviewer and highlights open risks	Lists the simulator signals that the camera test still uses

The agents follow clear guardrails. They have read access and can only propose; every finding is stored in the audit log next to the run; a small, fast language model is used for routine checks; and the gate remains the final decision. We will measure whether the agents help by injecting known failures, such as a broken contact model or a reward loophole, and recording how quickly each failure is detected with and without the agents.

12.3 Path to Real-World Deployment

A policy that passes the simulation gate is the start of a staged transfer, not the end. The planned sequence is: simulation qualification with realistic body collisions, a state estimator and an untouched final test; hardware in the loop with the robot's measured latency and friction; a harnessed robot on a test staircase with the same step heights as the strict test; supervised trials with a spotter and an emergency stop, where falls are logged against the simulation prediction; and finally shadow operation on real routes, with interventions per shift tracked. A pass at each stage is evidence for a risk assessment under ISO 10218, and later under the humanoid standard ISO 25785-1 (still a draft), rather than a certification in itself.

References

1. Kaczmarek, W., Łoty et al. (2021) 'Controlling an Industrial Robot Using a Graphic Tablet in Offline and Online Mode', *Sensors*, 21(7), 2439. doi: 10.3390/s21072439. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8036733/>
2. Szybicki, D. et al. (2022) 'Programming of Industrial Robots Using a Laser Tracker', *Sensors*, 22(17), 6464. doi: 10.3390/s22176464. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9460220/>
3. Pan, Z. et al. (2012) 'Recent progress on programming methods for industrial robots', *Robotics and Computer-Integrated Manufacturing*, 28(2), pp. 87–94. doi: 10.1016/j.rcim.2011.08.004. <https://www.sciencedirect.com/science/article/abs/pii/S0736584511001001>
4. Ragaglia, M. et al. (2016) 'Accurate sensorless lead-through programming for lightweight robots in structured environments', *Robotics and Computer-Integrated Manufacturing*, 39, pp. 9–21. doi: 10.1016/j.rcim.2015.11.002. <https://doi.org/10.1016/j.rcim.2015.11.002>.
5. Kolakowska, E. et al. (2014) 'Constraint optimization model of a scheduling problem for a robotic arm in automatic systems', *Robotics and Autonomous Systems*, 62(2), pp. 267–280. doi: 10.1016/j.robot.2013.09.005. <https://doi.org/10.1016/j.robot.2013.09.005>
6. Argall et al. (2009) 'A survey of robot learning from demonstration', *Robotics and Autonomous Systems*, 57(5), pp. 469–483. doi: 10.1016/j.robot.2008.10.024. <https://www.sciencedirect.com/science/article/abs/pii/S0921889008001772>
7. Ravichandar, H. et al. (2020) 'Recent Advances in Robot Learning from Demonstration', *Annual Review of Control, Robotics, and Autonomous Systems*, 3, pp. 297–330. doi: 10.1146/annurev-control-100819-063206. <https://www.annualreviews.org/content/journals/10.1146/annurev-control-100819-063206>
8. Zhu, Z. and Hu, H. (2018) 'Robot Learning from Demonstration in Robotic Assembly: A Survey', *Robotics*, 7(2), p. 17. doi: 10.3390/robotics7020017. <https://www.mdpi.com/2218-6581/7/2/17>
9. Hernandez Moreno, V. et al. (2024) 'Obstacles and opportunities for learning from demonstration in practical industrial assembly: A systematic literature review', *Robotics and Computer-Integrated Manufacturing*, 86, 102658. doi: 10.1016/j.rcim.2023.102658. <https://www.sciencedirect.com/science/article/pii/S0736584523001333>
10. Barekatin, A., Habibi, H. and Voos, H. (2024) 'A Practical Roadmap to Learning from Demonstration for Robotic Manipulators in Manufacturing', *Robotics*, 13(7), p. 100. doi: 10.3390/robotics13070100. <https://www.mdpi.com/2218-6581/13/7/100>
11. Si, W., Wang, N. and Yang, C. (2021) 'A review on manipulation skill acquisition through teleoperation-based learning from demonstration', *Cognitive Computation and Systems*, 3(1), pp. 1–16. doi: 10.1049/ccs2.12005. <https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/ccs2.12005>
12. Farajiparvar, P., Ying, H. and Pandya, A.P. (2020) 'A Brief Survey of Telerobotic Time Delay Mitigation', *Frontiers in Robotics and AI*, 7. doi: 10.3389/frobt.2020.578805. <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2020.578805/full>
13. González, C., Solanes, J.E., Muñoz, A., Gracia, L., Gírbés-Juan, V. and Tornero, J. (2021) 'Advanced teleoperation and control system for industrial robots based on augmented virtuality and haptic feedback', *Journal of Manufacturing Systems*, 59, pp. 283–298. doi: 10.1016/j.jmsy.2021.02.013. <https://www.sciencedirect.com/science/article/pii/S0278612521000492>
14. Wan, K., Li, C., Lo, F.S. and Zheng, P. (2024) 'A virtual reality-based immersive teleoperation system for remote human-robot collaborative manufacturing', *Manufacturing Letters*, 41, pp. 43–50. <https://www.sciencedirect.com/science/article/pii/S2213846324000658>
15. Zheng, C., Wang, K., Gao, S., Yu, Y., Wang, Z. and Tang, Y. (2025) 'Design of multi-modal feedback channel of human-robot cognitive interface for teleoperation in manufacturing', *Journal of Intelligent Manufacturing*, 36, pp. 4283–4303. doi: 10.1007/s10845-024-02451-x. <https://link.springer.com/article/10.1007/s10845-024-02451-x>
16. Ross, S., Gordon, G. and Bagnell, D. (2011) 'A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning', *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 15, pp. 627–635. <https://proceedings.mlr.press/v15/ross11a.html>
17. Liu, Z. et al. (2022) Robot learning towards smart robotic manufacturing: A review', *Robotics and Computer-Integrated Manufacturing*, 77, 102360. doi: 10.1016/j.rcim.2022.102360. <https://www.sciencedirect.com/science/article/pii/S0736584522000485>
18. Belkhale, S. et al. (2023) 'HYDRA: Hybrid Robot Actions for Imitation Learning', *Proceedings of the 7th Conference on Robot Learning*, 229, pp. 2113–2133. <https://proceedings.mlr.press/v229/belkhale23a.html>

19. Lu, Y. et al. (2022) 'AW-Opt: Learning Robotic Skills with Imitation and Reinforcement at Scale', Proceedings of the 5th Conference on Robot Learning, 164, pp. 1078–1088. <https://proceedings.mlr.press/v164/lu22a.html>
20. Kober, J. et al. (2013) 'Reinforcement learning in robotics: A survey', The International Journal of Robotics Research, 32(11), pp. 1238–1274. doi: 10.1177/0278364913495721. <https://doi.org/10.1177/0278364913495721>
21. Brunke, L. et al. (2022) 'Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning', Annual Review of Control, Robotics, and Autonomous Systems, 5, pp. 411–444. doi: 10.1146/annurev-control-042920-020211. <https://www.annualreviews.org/content/journals/10.1146/annurev-control-042920-020211>
22. Liu, P. et al. (2022) 'Robot Reinforcement Learning on the Constraint Manifold', Proceedings of the 5th Conference on Robot Learning, 164, pp. 1357–1366. <https://proceedings.mlr.press/v164/liu22c.html>
23. Zhao, W. et al. (2021) 'Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey', 2020 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 737–744. doi: 10.1109/SSCI47803.2020.9308468. <https://ieeexplore.ieee.org/document/9308468/>
24. Tang, C. et al. (2025) 'Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes', Annual Review of Control, Robotics, and Autonomous Systems, 8, pp. 153–188. doi: 10.1146/annurev-control-030323-022510. <https://www.annualreviews.org/content/journals/10.1146/annurev-control-030323-022510>
25. Muratore, F. et al. (2022) 'Robot Learning from Randomized Simulations: A Review', Frontiers in Robotics and AI, 9, 799893. doi: 10.3389/frobt.2022.799893. <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.799893/full>
26. Salvato, E. et al. (2021) 'Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning', IEEE Access, 9, pp. 153171–153187. doi: 10.1109/ACCESS.2021.3126658. https://ieeexplore.ieee.org/document/9606868?utm_source
27. Kadian, A., et al. (2024) 'A Survey on Sim-to-Real Transfer Methods for Robotic Manipulation', 2024 IEEE 22nd Jubilee International Symposium on Intelligent Systems and Informatics (SISY). doi: 10.1109/SISY62279.2024.10737545. https://ieeexplore.ieee.org/document/10737545?utm_source
28. 'The Reality Gap in Robotics: Challenges, Solutions, and Best Practices' (2025) Annual Review of Control, Robotics, and Autonomous Systems. <https://www.annualreviews.org/content/journals/10.1146/annurev-control-031924-100130>
29. Knox et al. (2009) 'Interactively Shaping Agents via Human Reinforcement', Proceedings of the Fifth International Conference on Knowledge Capture, pp. 9–16. doi: 10.1145/1597735.1597738. <https://dl.acm.org/doi/10.1145/1597735.1597738>
30. Griffith, S. et al. (2013) 'Policy Shaping: Integrating Human Feedback with Reinforcement Learning', Advances in Neural Information Processing Systems, 26. <https://proceedings.neurips.cc/paper/2013/hash/e034fb6b66aacc1d48f445ddfb08da98-Abstract.html>
31. Christiano, P.F. et al. (2017) 'Deep Reinforcement Learning from Human Preferences', Advances in Neural Information Processing Systems, 30. <https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html> <https://www.roboticsproceedings.org/rss14/p31.html>
32. Arzate Cruz, C. et al. (2020) 'A Survey on Interactive Reinforcement Learning: Design Principles and Open Challenges', Proceedings of the 2020 ACM Designing Interactive Systems Conference, pp. 2145–2158. doi: 10.1145/3393914.3394220. <https://dl.acm.org/doi/10.1145/3393914.3394220>
33. Brohan, A. et al. (2023) 'RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control', arXiv preprint arXiv:2307.15818. <https://arxiv.org/abs/2307.15818>
34. Open X-Embodiment Collaboration (2023) 'Open X-Embodiment: Robotic Learning Datasets and RT-X Models', 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 6892–6903. doi: 10.1109/ICRA48891.2024.10611477. <https://arxiv.org/abs/2310.08864>
35. Driess, D. et al. (2023) 'PaLM-E: An Embodied Multimodal Language Model', Proceedings of the 40th International Conference on Machine Learning, 202, pp. 8469–8488. <https://proceedings.mlr.press/v202/driess23a.html>
36. Kim, M.J. et al. (2024) 'OpenVLA: An Open-Source Vision-Language-Action Model', arXiv preprint arXiv:2406.09246. <https://arxiv.org/abs/2406.09246>
37. Schulman, J. et al. (2017) 'Proximal Policy Optimization Algorithms', arXiv preprint arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>
38. Rudin, N. et al. (2022) 'Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning', Proceedings of the 5th Conference on Robot Learning, 164, pp. 91–100. <https://proceedings.mlr.press/v164/rudin22a.html>
39. Zakka, K. et al. (2025) 'MuJoCo Playground', arXiv preprint arXiv:2502.08844. <https://arxiv.org/abs/2502.08844>

40. Todorov, E., Erez, T. and Tassa, Y. (2012) 'MuJoCo: A physics engine for model-based control', IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026–5033. doi: 10.1109/IROS.2012.6386109. <https://ieeexplore.ieee.org/document/6386109>
41. Freeman, C.D. et al. (2021) 'Brax – A Differentiable Physics Engine for Large Scale Rigid Body Simulation', arXiv preprint arXiv:2106.13281. <https://arxiv.org/abs/2106.13281>
42. Tobin, J. et al. (2017) 'Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World', IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 23–30. <https://arxiv.org/abs/1703.06907>
43. Lee, J. et al. (2020) 'Learning quadrupedal locomotion over challenging terrain', Science Robotics, 5(47), eabc5986. doi: 10.1126/scirobotics.abc5986. <https://www.science.org/doi/10.1126/scirobotics.abc5986>
44. Zhao, T.Z. et al. (2023) 'Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware', Robotics: Science and Systems. <https://arxiv.org/abs/2304.13705>
45. Chi, C. et al. (2023) 'Diffusion Policy: Visuomotor Policy Learning via Action Diffusion', Robotics: Science and Systems. <https://arxiv.org/abs/2303.04137>
46. Wilson, E.B. (1927) 'Probable Inference, the Law of Succession, and Statistical Inference', Journal of the American Statistical Association, 22(158), pp. 209–212. doi: 10.1080/01621459.1927.10502953. <https://doi.org/10.1080/01621459.1927.10502953>